

Universitat Politècnica de València
Master in Artificial Intelligence, Pattern Recognition and Digital Imaging
2022-2023

MACHINE TRANSLATION

3. Neural Machine Translation

Francisco Casacuberta

`fcn@prhlt.upv.es`

November 29, 2022

Index

- 1 Introduction ▷ 2
- 2 Synchronized feedforward neural networks ▷ 11
- 3 Synchronized recurrent neural networks ▷ 14
- 4 Neural machine translation ▷ 23
- 5 Attention models and recurrent neural networks ▷ 31
- 6 Attention models for all ▷ 40
- 7 Training and Decoding with NMT ▷ 59
- 8 Translation results with neural machine translation ▷ 67
- 9 Other issues of NMT ▷ 73
- 10 Bibliography ▷ 84

Index

- 1 *Introduction* ▷ 2
- 2 Synchronized feedforward neural networks ▷ 11
- 3 Synchronized recurrent neural networks ▷ 14
- 4 Neural machine translation ▷ 23
- 5 Attention models and recurrent neural networks ▷ 31
- 6 Attention models for all ▷ 40
- 7 Training and Decoding with NMT ▷ 59
- 8 Translation results with neural machine translation ▷ 67
- 9 Other issues of NMT ▷ 73
- 10 Bibliography ▷ 84

Introduction

- Traditional approach to MT: Discrete representation of words and sentences.
- Most techniques of machine learning are developed in continuous spaces (i.e. vector spaces)
- In machine learning, (deep) neural networks are good models for many applications.
- Can words and sentences be represented in a continuous space? **Word and sentence embeddings.**
- Can neural networks deal with sequences? **Dynamic feed-forward networks and recurrent neural networks.**

Learned word embeddings

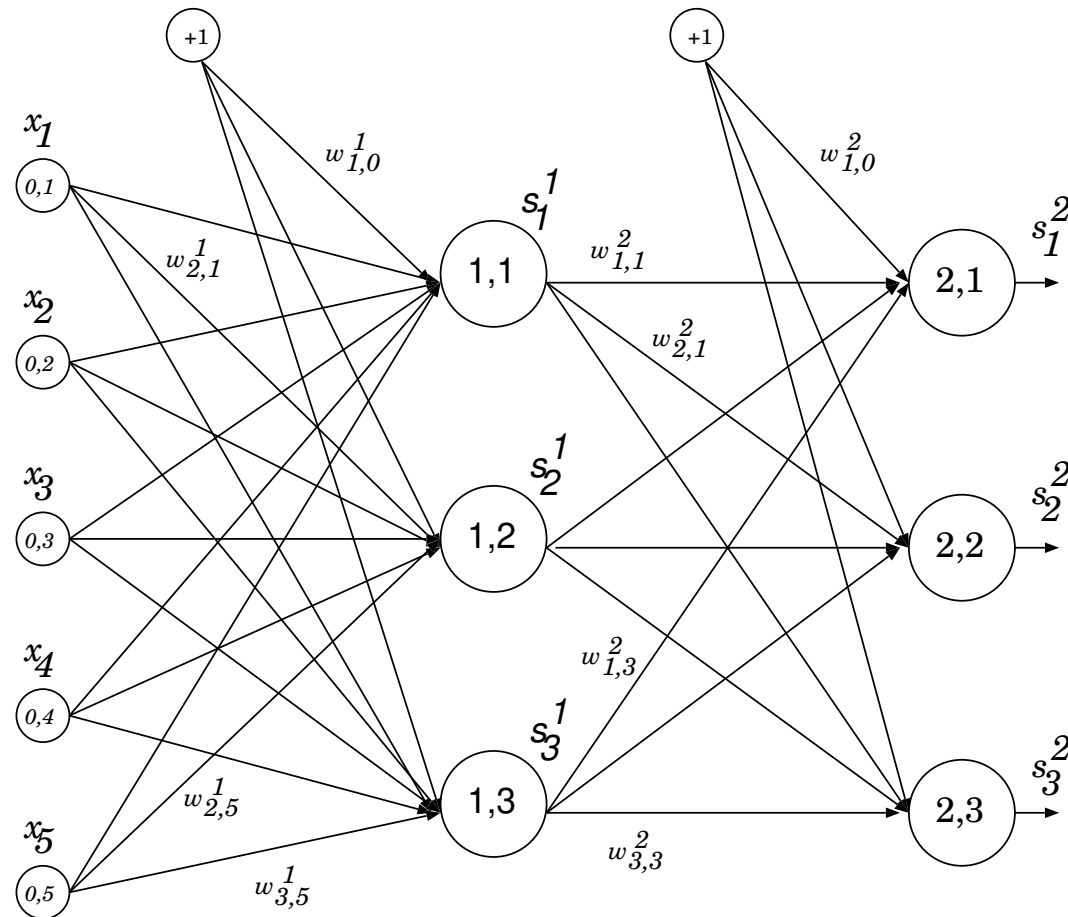
Problem: From words to vectors.

- ***Local (one-hot) coding*** and distributed coding.
- Neural language model (Bengio 2003)
- Bag-of-words neural networks (Mikolov 2013)
- Continuous skip-grams (Mikolov 2013)
- Pre-trained neural networks: BERT (Devlin 2019)
- ...
- Extension to sentence embeddings: SentenceBERT, Universal Sentence Encoder, ...

Byte-Pair Encoding (BPE) (Sennrich 2015)

- Problem: Word embedding of unseen words.
- Solution: The use of subword units learned from a training set.
- *Byte Pair Encoding* (BPE) is a data compression technique that iteratively replaces the most frequent pair of bytes in a sequence with a single, unused byte.
- Toolkit: <https://github.com/rsennrich/subword-nmt>
- Other related techniques: [SentencePiece](#), ...
- Character based units: convolutional networks.

A multilayer perceptron



Hidden layer

$$s^1 = \mathbf{f}(\mathbf{W}^1 \mathbf{x})$$

Output layer

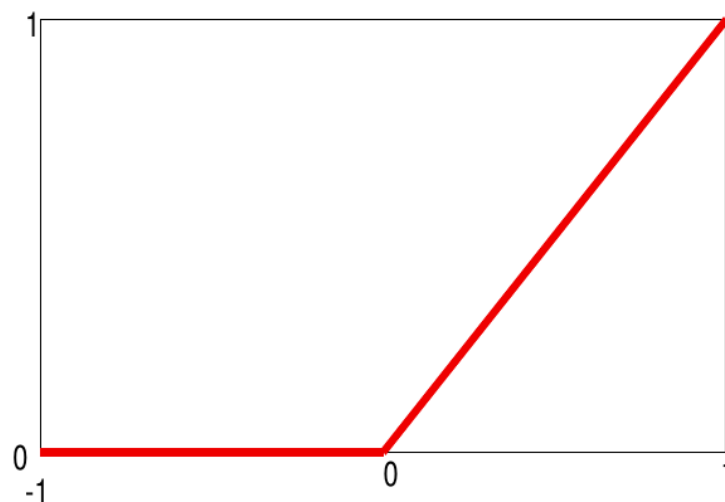
$$s^2 = \mathbf{f}(\mathbf{W}^2 s^1)$$

Activation functions

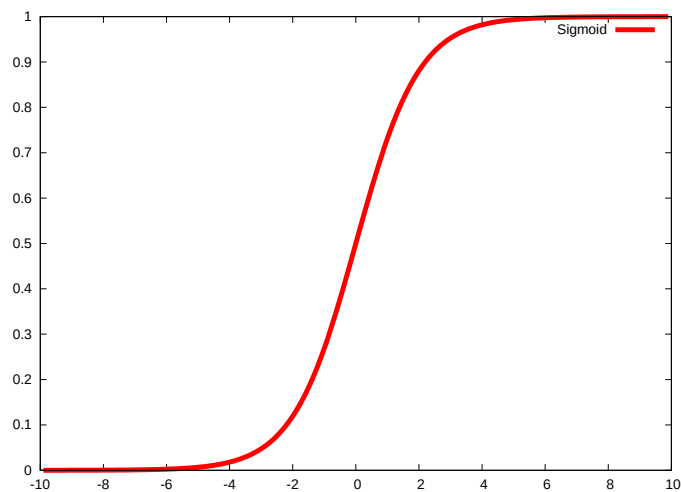
$$\mathbf{f}(\mathbf{z}) = (f(z_1), \dots, f(z_d))^t \text{ for } \mathbf{z} \equiv (z_1, \dots, z_d)^t \in \mathbb{R}^d$$

- **ReLU** (rectified linear unit): $f_R(z_j) = \max(0, z_j)$
- **Sigmoid**: $f_S(z_j) = \frac{1}{1 + \exp(-z_j)}$
- **Hyperbolic tangent**: $f_T(z_j) = \frac{\exp(z_j) - \exp(-z_j)}{\exp(z_j) + \exp(-z_j)} \quad (f_T(z_j) = 2 f_S(2 z_j) - 1)$
- **Softmax**: $f_{SM}(z_j) = \frac{\exp(z_j)}{\sum_{j'} \exp(z_{j'})} \quad \left(f_{SM}(z_j) = f_S(z_j - \ln(\sum_{j' \neq j} \exp(z_{j'}))) \right)$
- PReLU, ELU, Maxout, ...

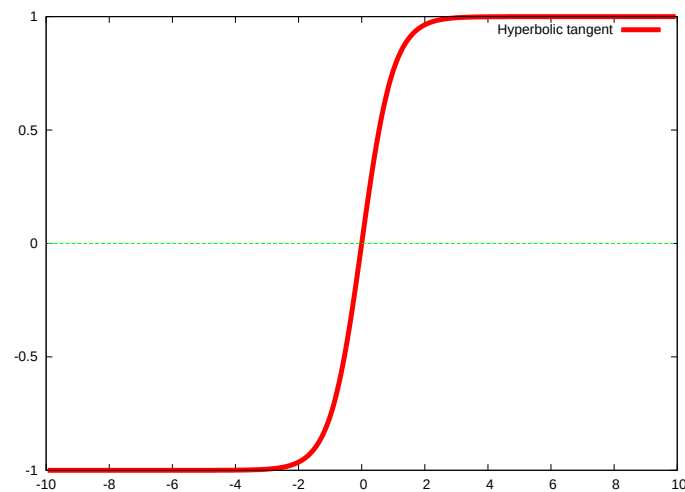
Activation functions



ReLu



Sigmoid



Hyperbolic tangent

The softmax activation function

A problem with the use of softmax in training

$$f_{sm}(z_w) = \frac{\exp(z_w)}{\sum_{w'} \exp(z_{w'})}$$

The sum in the normalization factor is extended on all the vocabulary $|V|$.

Solutions:

- Hierarchical softmax: using a binary tree representation of the output layer (Huffman coding)
- Negative sampling: each training sample only modify a subset of the weights (the weights of a small subset of “negative” words).
- Subsampling of frequent words.

Training neural networks

- A neural network defines $\mathbf{f} : \mathbb{R}^{D_X} \rightarrow \mathbb{R}^{D_Y} : \mathbf{f}(\mathbf{x}_n; \mathbf{W}) = \mathbf{y}_n$ for $1 \leq n \leq N$.
- A **training sequence** $T = (\mathbf{x}_1, \mathbf{t}_1), \dots, (\mathbf{x}_N, \mathbf{t}_N) : \mathbf{x}_n \in \mathbb{R}^{D_X}, \mathbf{t}_n \in \mathbb{R}^{D_Y}$
- **Total error** (regression) is:
$$\mathcal{F}_T(\mathbf{W}) = \sum_{n=1}^N \frac{1}{2} \sum_{i=1}^{D_Y} (t_{n,i} - y_{n,i})^2$$
- **Cross-entropy loss** (classification) ($\mathbf{t}_n \in \{0, 1\}^{D_Y}$):
$$\mathcal{F}_T(\mathbf{W}) = - \sum_{n=1}^N \sum_{i=1}^{D_Y} t_{n,i} \log y_{n,i}$$
- Training goal: $\widehat{\mathbf{W}} = \underset{\mathbf{W}}{\operatorname{argmin}} \mathcal{F}_T(\mathbf{W})$
- Computing a local minimum of $\mathcal{F}$: **GRADIENT DESCENT**

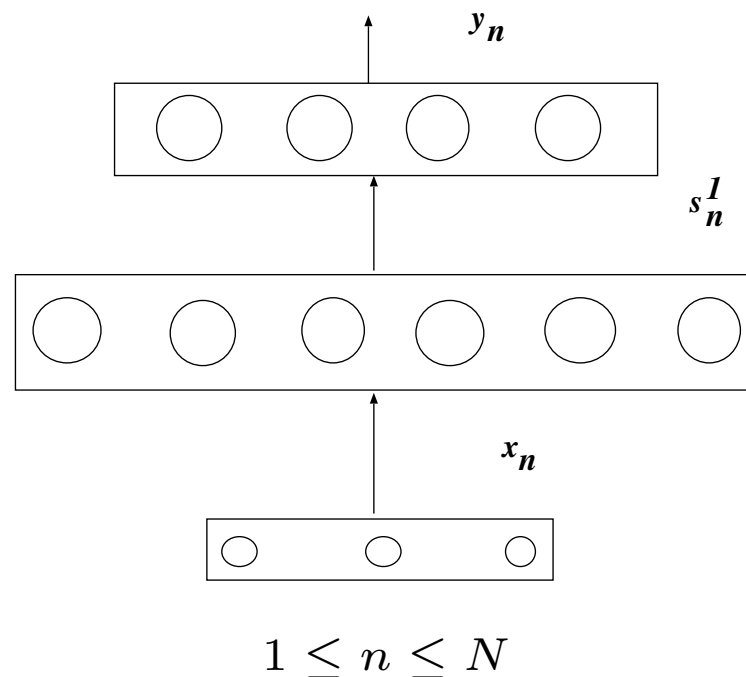
$$\Delta \mathbf{W} = -\rho \nabla_{\mathbf{W}} \mathcal{F}_T(\mathbf{W})$$

Index

- 1 Introduction ▷ 2
- 2 *Synchronized feedforward neural networks* ▷ 11
- 3 Synchronized recurrent neural networks ▷ 14
- 4 Neural machine translation ▷ 23
- 5 Attention models and recurrent neural networks ▷ 31
- 6 Attention models for all ▷ 40
- 7 Training and Decoding with NMT ▷ 59
- 8 Translation results with neural machine translation ▷ 67
- 9 Other issues of NMT ▷ 73
- 10 Bibliography ▷ 84

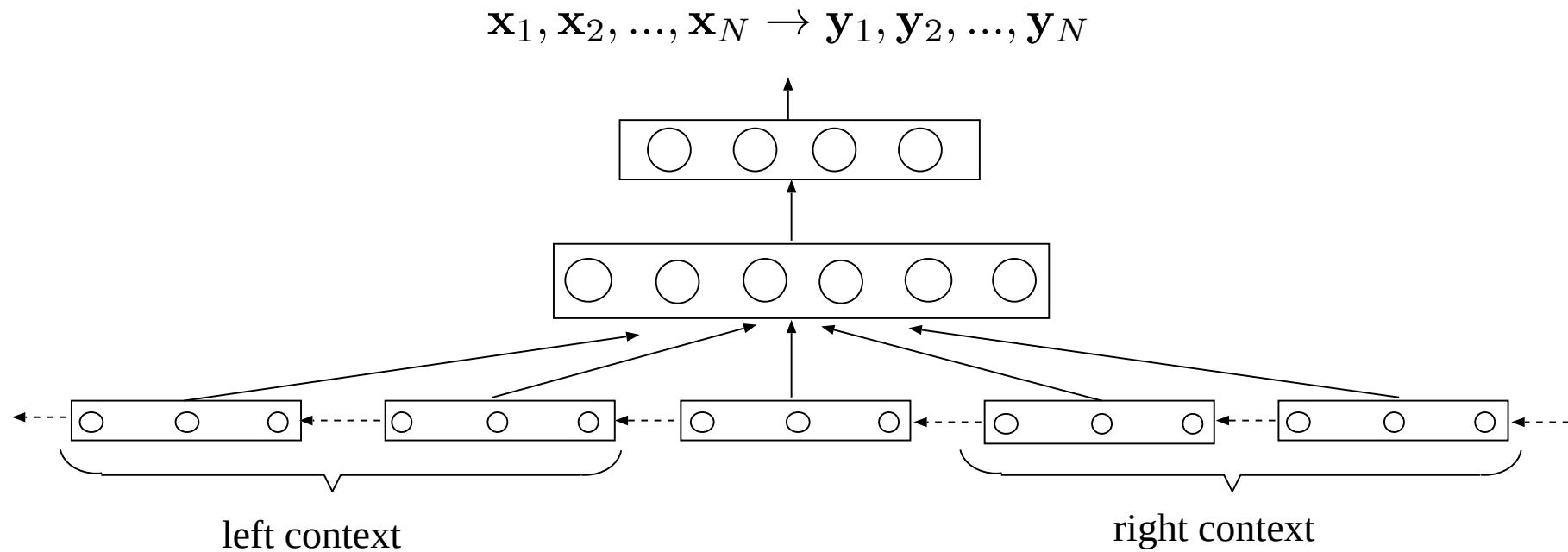
Sequence processing with dynamic feedforward networks

$$\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N \rightarrow \mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_N$$



t	input layer		hidden layer		output layer
1	$\mathbf{x}_1$	$\Rightarrow$	$\mathbf{s}_1^1$	$\Rightarrow$	$\mathbf{y}_1$
2	$\mathbf{x}_2$	$\Rightarrow$	$\mathbf{s}_2^1$	$\Rightarrow$	$\mathbf{y}_2$
			...		
N	$\mathbf{x}_N$	$\Rightarrow$	$\mathbf{s}_N^1$	$\Rightarrow$	$\mathbf{y}_N$

Sequence processing with dynamic feedforward networks



$$1 \leq n \leq N$$

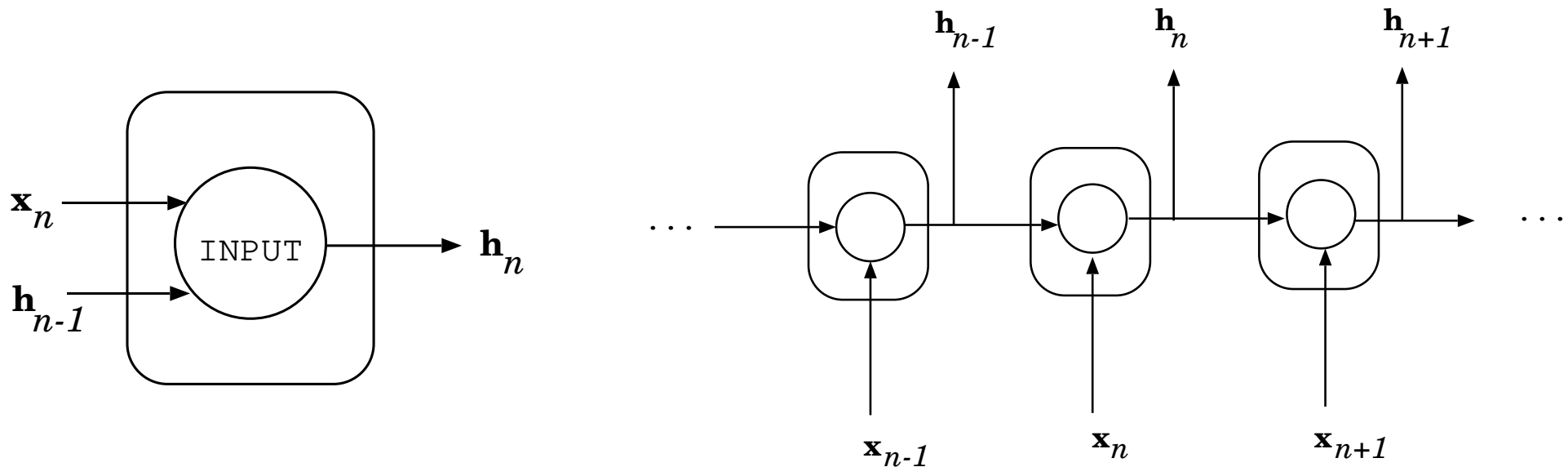
t	input layer		hidden layer		output layer
1	$\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2$	$\Rightarrow$	$\mathbf{s}_1^1$	$\Rightarrow$	$\mathbf{y}_1$
2	$\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$	$\Rightarrow$	$\mathbf{s}_2^1$	$\Rightarrow$	$\mathbf{y}_2$
			...		
N	$\mathbf{x}_{N-1}, \mathbf{x}_N, \mathbf{x}_{N+1}$	$\Rightarrow$	$\mathbf{s}_N^1$	$\Rightarrow$	$\mathbf{y}_N$

Index

- 1 Introduction ▷ 2
- 2 Synchronized feedforward neural networks ▷ 11
- 3 *Synchronized recurrent neural networks* ▷ 14
- 4 Neural machine translation ▷ 23
- 5 Attention models and recurrent neural networks ▷ 31
- 6 Attention models for all ▷ 40
- 7 Training and Decoding with NMT ▷ 59
- 8 Translation results with neural machine translation ▷ 67
- 9 Other issues of NMT ▷ 73
- 10 Bibliography ▷ 84

Simple recurrent networks

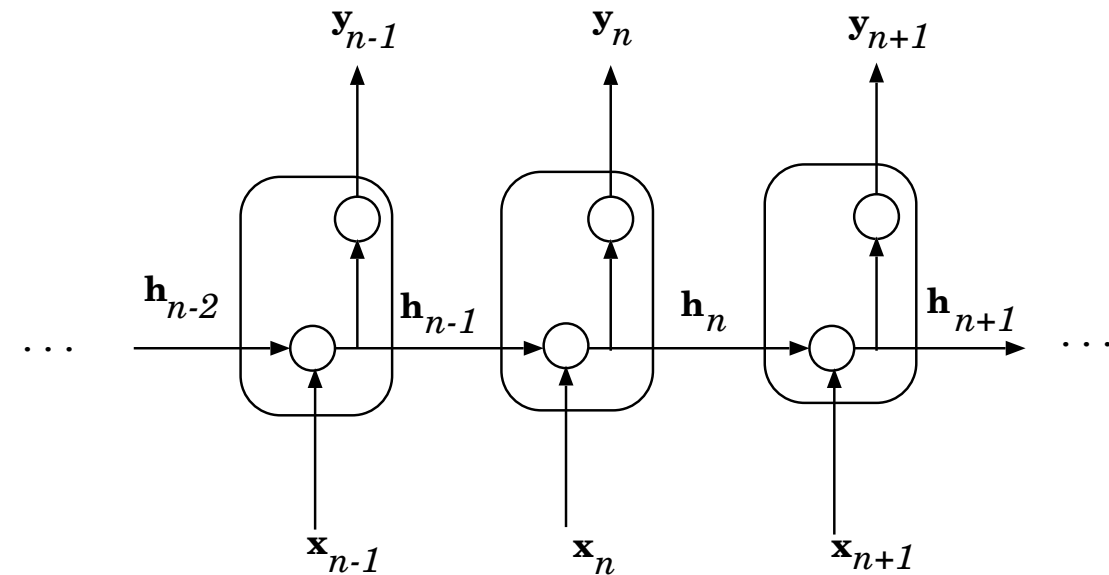
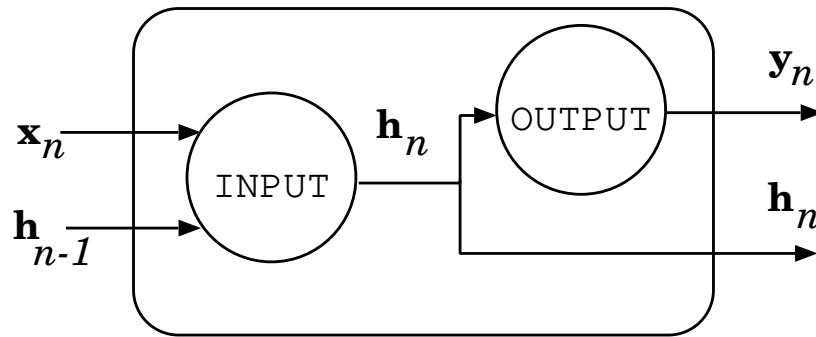
- A set H of D_H units and a set X of D_X inputs.
- Input: $\mathbf{x}_n \in \mathbb{R}^{D_X}$ and output: $\mathbf{h}_n \in \mathbb{R}^{D_H}$, $n = 1, 2, \dots$
- $\mathbf{y}_n = \mathbf{h}_n = \mathbf{f}(\mathbf{W}_X \mathbf{x}_n + \mathbf{W}_H \mathbf{h}_{n-1}) = \mathbf{F}(\mathbf{x}_n, \mathbf{h}_{n-1})$ with $\mathbf{h}_0 = \mathbf{0}$



Unfolding a simple recurrent network

Elman recurrent neural networks

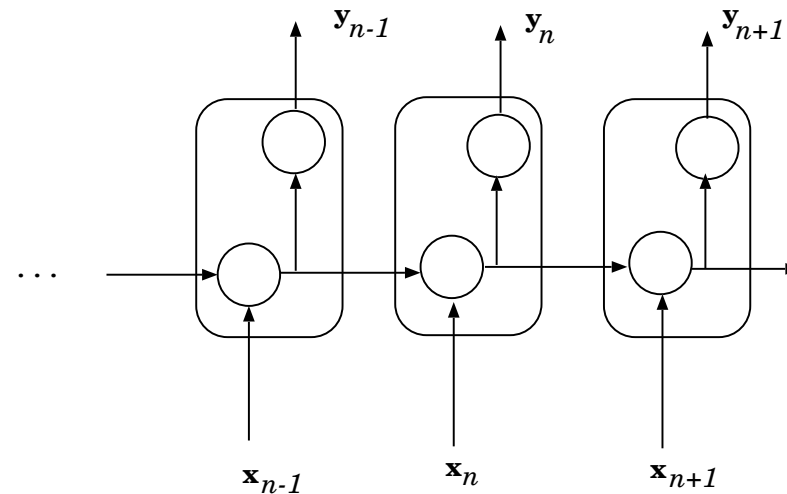
- A set Y of D_Y units, a set X of D_X inputs and a set H of D_H hidden units
- Input: $\mathbf{x}_n \in \mathbb{R}^{D_X}$, output: $\mathbf{y}_n \in \mathbb{R}^{D_Y}$, and hidden states: $\mathbf{h}_n \in \mathbb{R}^{D_H}$, $n = 1, 2, \dots$
- $\mathbf{y}_n = \mathbf{f}(\mathbf{W}_Y \mathbf{h}_n)$ and $\mathbf{h}_n = \mathbf{f}(\mathbf{W}_H \mathbf{h}_{n-1} + \mathbf{W}_X \mathbf{x}_n)$ with $\mathbf{h}_0 = \mathbf{0}$



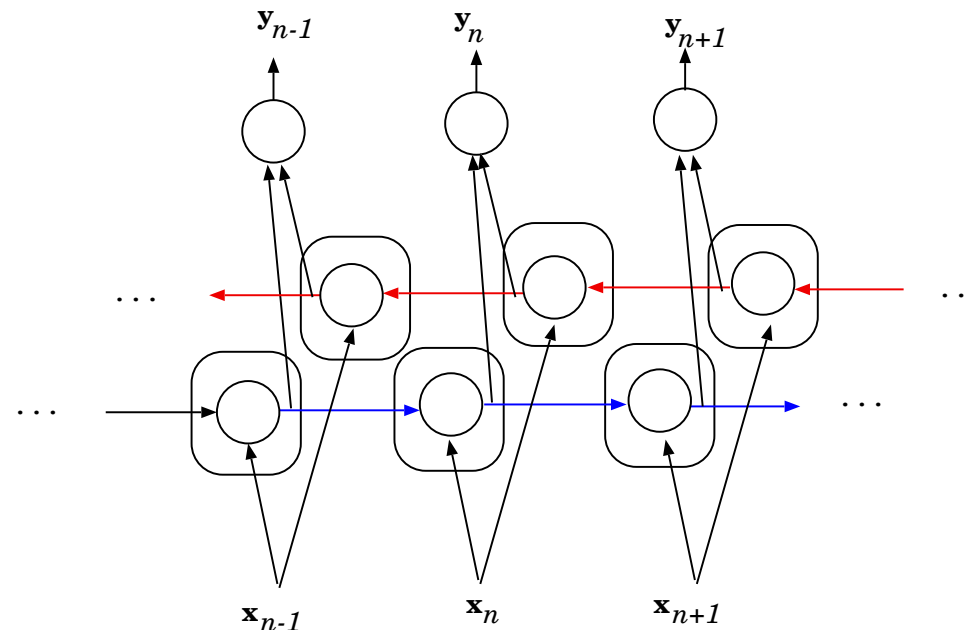
Unfolding Elman network

Bidirectional recurrent neural networks (Schuster & Paliwal 1997)

Dependencies
with the past
inputs, but with the
future?



Standard dynamics



Bidirectional dynamics

Bidirectional recurrent neural networks

- A set of D_H forward units, a set of D_H backward units, a set of D_Y output units and a set of D_X inputs.
- Input: $\mathbf{x}_n \in \mathbb{R}^{D_X}$ and output: $\mathbf{y}_n \in \mathbb{R}^{D_Y}$.
- **Forward:** $\mathbf{h}_n^f = \mathbf{f}(\mathbf{W}_H^f \mathbf{h}_{n-1}^f + \mathbf{W}_X^f \mathbf{x}_n)$ $n = 1, 2, \dots, N$ with $\mathbf{h}_0 = \mathbf{0}$
- **Backward:** $\mathbf{h}_n^b = \mathbf{f}(\mathbf{W}_H^b \mathbf{h}_{n+1}^b + \mathbf{W}_X^b \mathbf{x}_n)$ $n = N, N-1, \dots, 1$ with $\mathbf{h}_{N+1} = \mathbf{0}$
- **Output:** $\mathbf{y}_n = \mathbf{f}(\mathbf{W}_Y^f \mathbf{h}_n^f + \mathbf{W}_Y^b \mathbf{h}_n^b)$ $n = 1, 2, \dots, N$
- **Output** (alternative): $\mathbf{y}_n = \mathbf{f}(\mathbf{W}_Y[\mathbf{h}_n^f, \mathbf{h}_n^b])$ $n = 1, 2, \dots, N$

Training recurrent neural networks

- Back-propagation through time algorithm (BPTT)
- Back-propagation through time algorithm with momentum.
- Truncate gradient with momentum.

Text translation using Elman networks

(Castaño et al. Eurospeech 1997)

- **Problem:** translate a source sentence from a source language to a sentence from a target language in a hotel-desk scenario:
/¿Tiene habitaciones libres?/ $\Rightarrow$ /Do you have any room available?/
- **Topology**
 - Elman network
 - Distributed coding:
 - * 61 input units (132 words)
 - * 52 output units (87 words)
 - * 160 hidden units
 - windows of 6 words
- **Training**
 - Truncate gradiente with momentum and pattern-based training
 - Training corpus: 5000 labeled pairs
 - Number of iterations: 100.
- **Test**
 - 1000 pairs (12.6 words / source sentence and 11.8 words / output sentence).
 - Percentage of right translated sentences: 49.5%
 - Percentage of right translated words: 93.1%

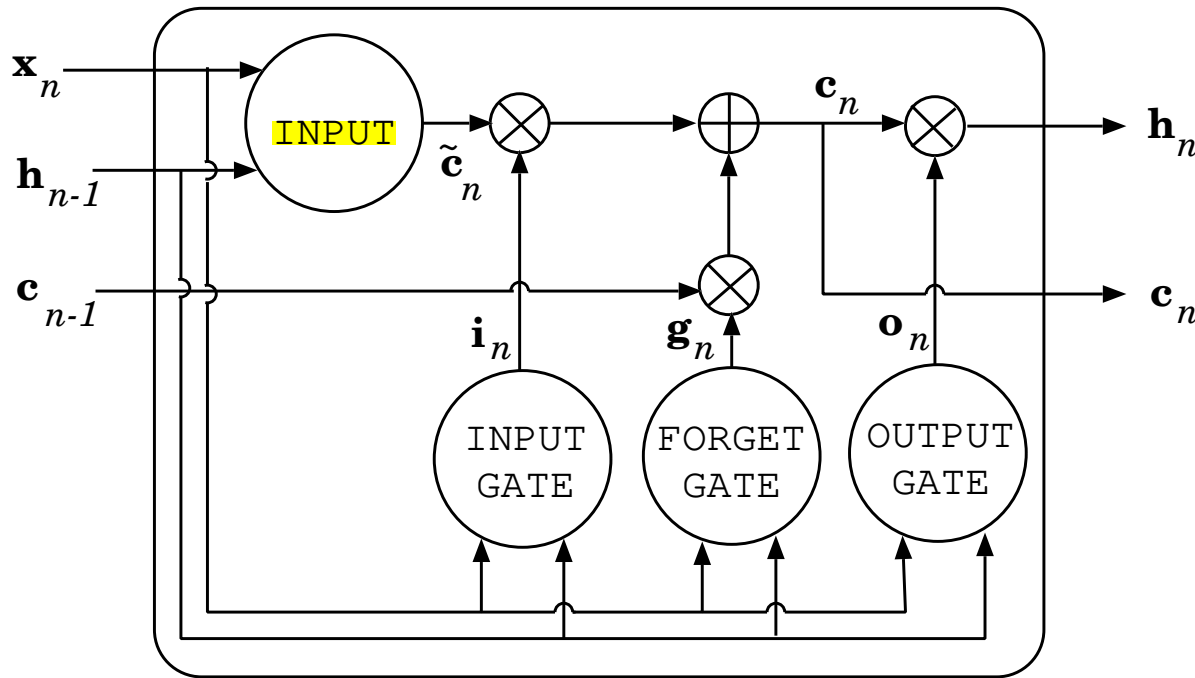
Long Short-Term Memory (LSTM) [Hochreiter 1997]

Problems with recurrent neural networks:

- The output depends on the complete past information and sometimes only recent information is needed and sometimes more far context is needed
- In back-propagation through time algorithm and exact gradient or real-time recurrent learning algorithms the errors that propagated backwards in time tend to vanish or to oscillate.

A solution: Long Short-Term Memory (LSTM) or Gated Recurrent Units (GRU)

Long Short-Term Memory (LSTM)



- $\mathbf{i}_n = \mathbf{f}_s(\mathbf{W}_Y^I \mathbf{y}_{n-1} + \mathbf{W}_X^I \mathbf{x}_n)$
- $\mathbf{g}_n = \mathbf{f}_s(\mathbf{W}_Y^F \mathbf{y}_{n-1} + \mathbf{W}_X^F \mathbf{x}_n)$
- $\mathbf{o}_n = \mathbf{f}_s(\mathbf{W}_Y^O \mathbf{y}_{n-1} + \mathbf{W}_X^O \mathbf{x}_n)$
- $\tilde{\mathbf{c}}_n = \mathbf{f}_{th}(\mathbf{W}_Y^C \mathbf{y}_{n-1} + \mathbf{W}_X^C \mathbf{x}_n)$
- $\mathbf{c}_n = \mathbf{g}_n \times \mathbf{c}_{n-1} + \mathbf{i}_n \times \tilde{\mathbf{c}}_n$
- $\mathbf{y}_n = \mathbf{o}_n \times \mathbf{f}_{th}(\mathbf{c}_n)$

$$\mathbf{y}_n = \mathbf{F}(\mathbf{x}_n, \mathbf{y}_{n-1})$$

Index

- 1 Introduction ▷ 2
- 2 Synchronized feedforward neural networks ▷ 11
- 3 Synchronized recurrent neural networks ▷ 14
- 4 *Neural machine translation* ▷ 23
- 5 Attention models and recurrent neural networks ▷ 31
- 6 Attention models for all ▷ 40
- 7 Training and Decoding with NMT ▷ 59
- 8 Translation results with neural machine translation ▷ 67
- 9 Other issues of NMT ▷ 73
- 10 Bibliography ▷ 84

Neural Machine Translation

- Early proposals with synchronized recurrent neural networks (Castaño 1997)
- For a source sentence x_1^J search for a target sentence $\hat{y}_1^{\hat{I}}$,

$$\begin{aligned}\hat{y}_1^{\hat{I}} &= \operatorname{argmax}_{I, y_1^I} p(y_1^I \mid x_1^J) \\ &= \operatorname{argmax}_{I, y_1^I} \prod_{i=1}^I p(y_i \mid y_1^{i-1}, x_1^J) \\ &= \operatorname{argmax}_{I, y_1^I} \prod_{i=1}^I p(y_i \mid y_1^{i-1}, u(x_1^J))\end{aligned}$$

where $u(x_1^J)$ is a representation of x_1^J

Neural Machine Translation

- Non synchronized neural networks (many available tool-kits): The **encoder-decoder approach (+ attention models)** based on
 - LSTMs or **GRUs** + cross-attention model.
 - **Transformer** (attention models for everything).
 - Convolutional neural networks and/or recurrent neural networks (hybrid).
- MT applications:
 - Text and speech translation.
 - Interactive MT, multimodal MT, automatic post-editing, modernization of old text transcriptions, ...
- Other applications:
 - Automatic summarization.
 - Image description, video description, multilingual image description, visual question answering, image generation,
 - ...

Encoder-decoder approach to NMT (Peris 2018)

1. Project source words to a sequence of continuous vectors → Source word embeddings.
2. Generate a **contextual representation** of the source sentence → Encoder NN.
3. Generate an a-posteriori probabilistic distributions of successive target words → Decoder NN.
4. Encode target words → Target word embeddings.

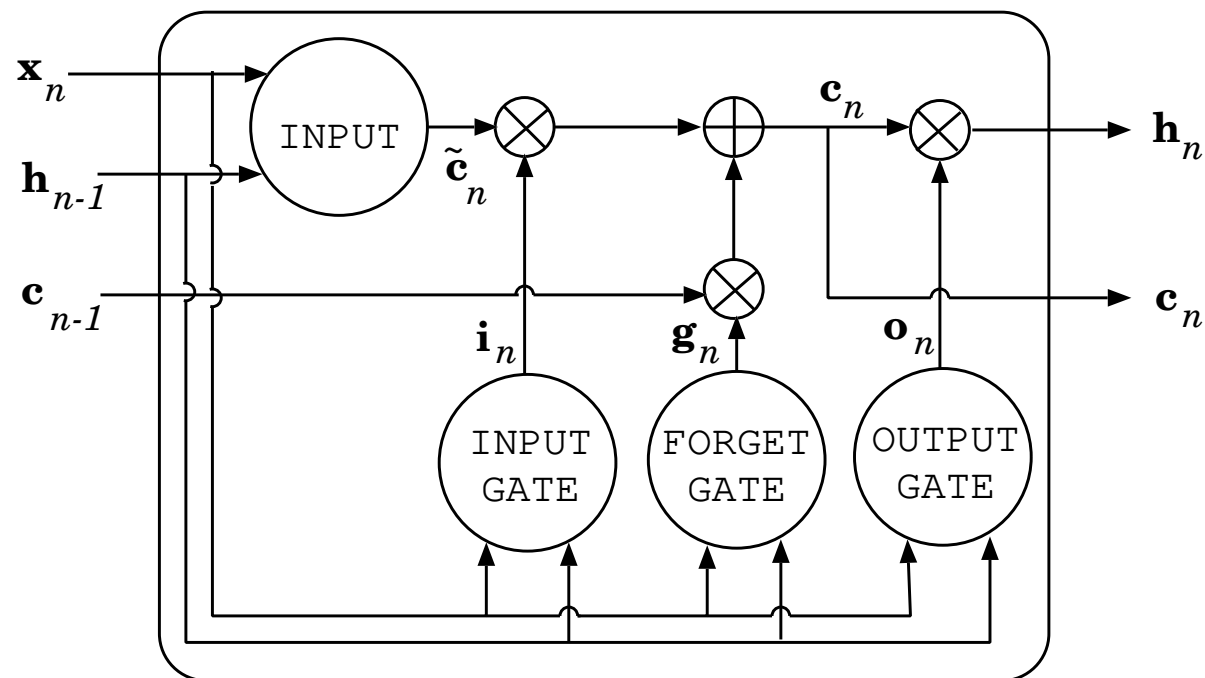
Encoder-decoder with LSTMs only

- Goal: Given a source sentence x_1^J and a target sentence y_1^I , compute:

$$p(y_1^I \mid x_1^J) = \prod_{i=1}^I p(y_i \mid y_1^{i-1}, u(x_1^J))$$

- Neural networks: LSTMs or GRUs

Long Short-Term Memory (LSTM) (Hochreiter & Schmidhuber, 1997)



$$\mathbf{h}_j^e = \mathbf{F}(\mathbf{x}_j, \mathbf{h}_{j-1}^e) \quad 1 \leq j \leq J$$

$$\mathbf{h}_i^d = \mathbf{F}(\mathbf{y}_{i-1}, \mathbf{h}_{i-1}^d) \quad 1 \leq i \leq I$$

Encoder-decoder approach to NMT (Bahdanau et al. 2015)

- The RNN-based encoder (F_e) converts a source word sequence $x_1, \dots, x_J \equiv x_1^J$ into a vector $\mathbf{u} = \mathbf{u}(x_1^J)$,

$$\mathbf{h}_0^e = \mathbf{0}$$

$$\mathbf{h}_j^e = F_e(\mathbf{W}_E(x_j), \mathbf{h}_{j-1}^e) = F_e(\mathbf{x}_j, \mathbf{h}_{j-1}^e) \quad 1 \leq j \leq J$$

$$\mathbf{u} = u(\mathbf{h}_1^e, \dots, \mathbf{h}_J^e)$$

where $\mathbf{h}_j^e$ is the states of a RNN-encoder and $\mathbf{u}$ could be $\mathbf{h}_J^e$ (sentence embedding)

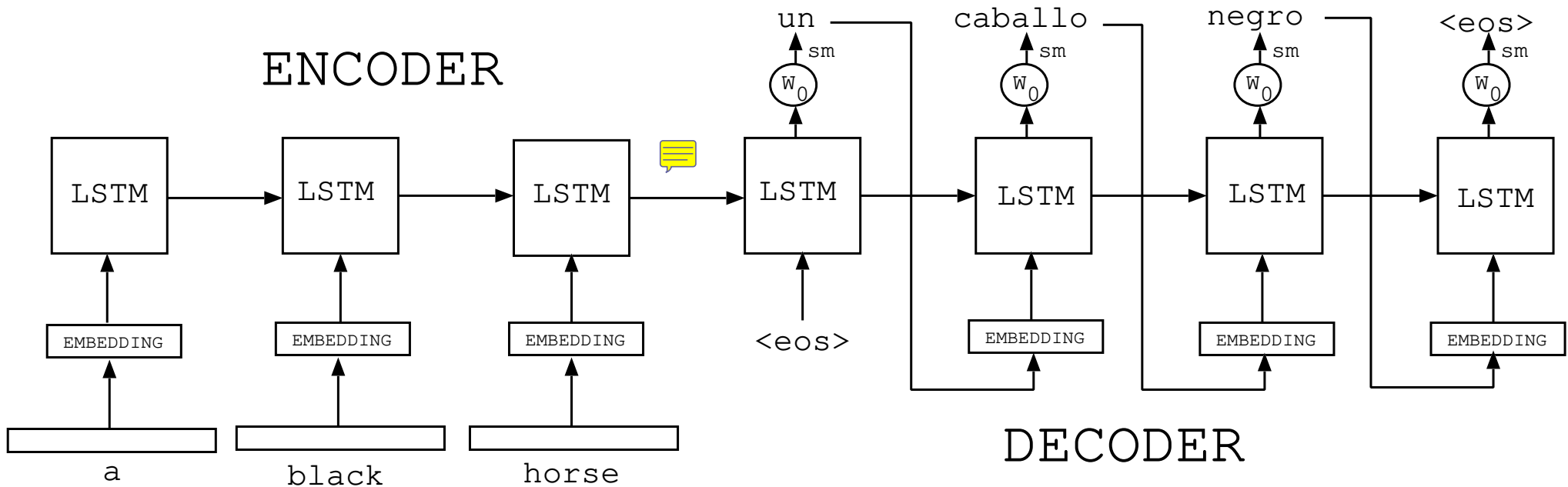
- The RNN-based decoder (F_d) is a kind of target language model

$$\mathbf{h}_0^d = \mathbf{u} \equiv \mathbf{h}_J^e$$

$$\mathbf{h}_i^d = F_d(\mathbf{W}_E(y_{j-1}), \mathbf{h}_{i-1}^d) = F_d(\mathbf{y}_{j-1}, \mathbf{h}_{i-1}^d) \quad 1 \leq i \leq I$$

$$p(y_1^I \mid x_1^J) = \prod_{i=1}^I p(y_i \mid y_1^{i-1}, u(x_1^J)) = \prod_{i=1}^I \mathbf{f}_{sm}(\mathbf{W}_O \mathbf{h}_i^d)_{i(y_i)}$$

Text translation using LSTM (Sutskever 2014)



Experiments:
WMT14 English-to-French

Systems	BLEU
Baseline (PB+Neural LM)	33.3
Ensemble of 5 LSTMs	34.8
Reescoring 1000-best with ensemble of 5 LSTMs	36.5
Moses for WMT14	37.0

Index

- 1 Introduction ▷ 2
- 2 Synchronized feedforward neural networks ▷ 11
- 3 Synchronized recurrent neural networks ▷ 14
- 4 Neural machine translation ▷ 23
- 5 *Attention models and recurrent neural networks* ▷ 31
- 6 Attention models for all ▷ 40
- 7 Training and Decoding with NMT ▷ 59
- 8 Translation results with neural machine translation ▷ 67
- 9 Other issues of NMT ▷ 73
- 10 Bibliography ▷ 84

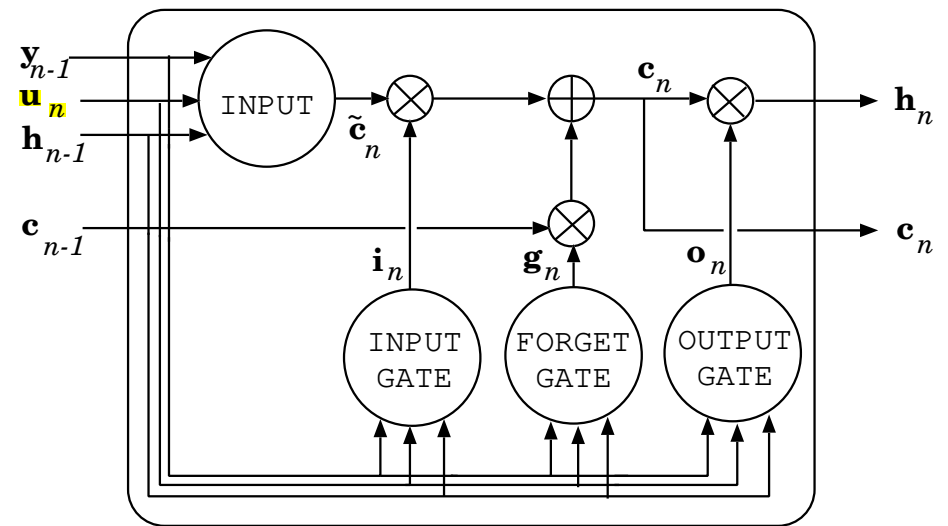
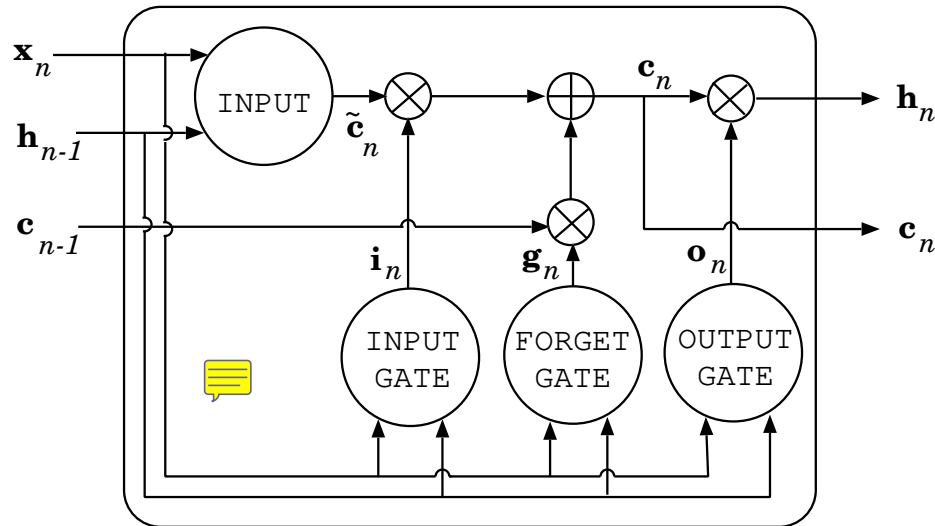
Encoder-decoder with LSTMs (Vaswani 2017)(Ney 2018)

- Goal: Given a source sentence x_1^J and a target sentence y_1^I , compute:

$$p(y_1^I \mid x_1^J) = \prod_{i=1}^I p(y_i \mid y_1^{i-1}, u(x_1^J))$$

- Neural networks: LSTMs or GRUs.
- **Intra-sentence attention** (i, j) : For each i , $1 \leq i \leq I$ a probabilistic contribution of j , $1 \leq j \leq J$.

Long Short-Term Memory (LSTM)



$$\mathbf{h}_j^e = \mathbf{F}(\mathbf{x}_j, \mathbf{h}_{j-1}^e) \quad 1 \leq j \leq J$$

$$\mathbf{h}_i^d = \mathbf{F}(\mathbf{y}_{i-1}, \mathbf{h}_{i-1}^e, \mathbf{u}_i) \quad 1 \leq i \leq I$$

Attention model (Vaswani 2018)

Given a sequence of encoder states $\mathbf{h}_j^e$ for $1 \leq j \leq J$ and a decoder state $\mathbf{h}_{i-1}^d$ for $1 \leq i \leq I$ an **attention model** for the decoder state $\mathbf{h}_i^d$ is a function:

$$\mathbf{u}_i = \mathbf{a}(\mathbf{h}_1^e, \dots, \mathbf{h}_J^e, \mathbf{h}_{i-1}^d)$$

where $\mathbf{a}$ is:

$$\mathbf{u}_i = \sum_{j=1}^J f_j(\mathbf{h}_1^e, \dots, \mathbf{h}_J^e, \mathbf{h}_{i-1}^d) \mathbf{W}_V \mathbf{h}_j^e$$

f_j is a softmax:

$$f_j(\mathbf{h}_1^e, \dots, \mathbf{h}_J^e, \mathbf{h}_{i-1}^d) = \frac{\exp(\mathbf{W}_Q \mathbf{h}_{i-1}^d \cdot \mathbf{W}_K \mathbf{h}_j^e)}{\sum_{j'=1}^J \exp(\mathbf{W}_Q \mathbf{h}_{i-1}^d \cdot \mathbf{W}_K \mathbf{h}_{j'}^e)} \quad 1 \leq j \leq J$$

NMT with attention model (Bahdanau et al. 2015) (Luong et al. 2015)

- The RNN-based encoder:

$$\mathbf{h}_j^e = \mathbf{F}_e(\mathbf{W}_E(x_j), \mathbf{h}_{j-1}^e) = \mathbf{F}_e(\mathbf{x}_j, \mathbf{h}_{j-1}^e) \quad 1 \leq j \leq J \text{ and } \mathbf{h}_0^e = \mathbf{0}$$

- The RNN-based decoder with attention model:

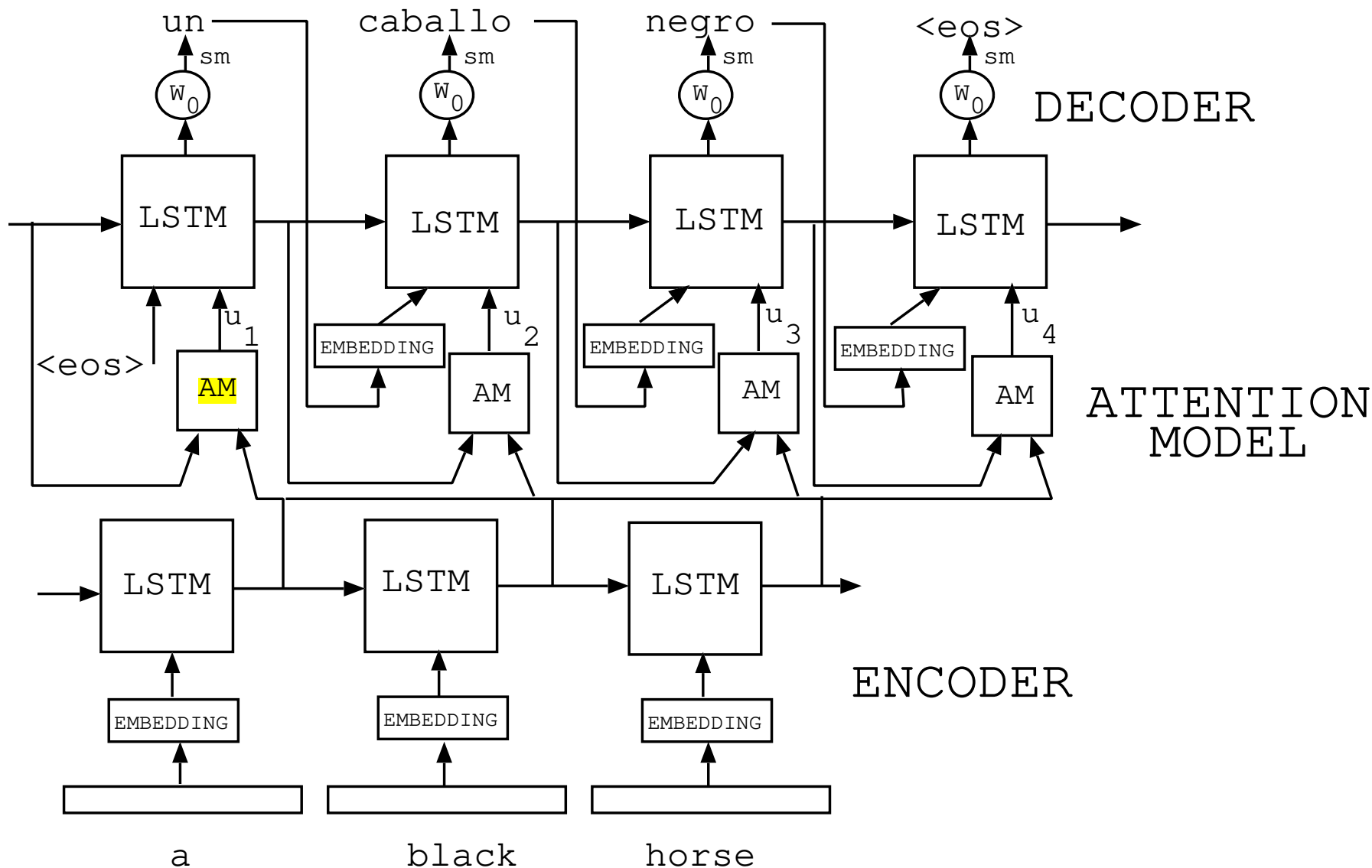
$$\mathbf{u}_i = \mathbf{a}(\mathbf{h}_1^e, \dots, \mathbf{h}_J^e, \mathbf{h}_{i-1}^d) \quad 1 \leq i \leq I$$

$$\mathbf{h}_i^d = \mathbf{F}_d(\mathbf{W}_E(y_{i-1}), \mathbf{h}_{i-1}^d, \mathbf{u}_i) = \mathbf{F}_d(\mathbf{y}_{i-1}, \mathbf{h}_{i-1}^d, \mathbf{u}_i) \quad 1 \leq i \leq I \text{ and } \mathbf{h}_0^d = \mathbf{0}$$

- Output generation:

$$p(y_1^I \mid x_1^J) = \prod_{i=1}^I p(y_i \mid y_1^{i-1}, u(x_1^J)) = \prod_{i=1}^I \mathbf{f}_{sm}(\mathbf{W}_O \mathbf{h}_i^d)_{i(y_i)}$$

Text translation with attention mechanism (Luong 2015)



Bidirectional RNN for neural machine translation (Bahdanau et al. 2015)

- The RNN-based encoder:



$$\mathbf{h}_j^{e,f} = \mathbf{F}_e(\mathbf{x}_j, \mathbf{h}_{j-1}^{e,f}) \quad 1 \leq j \leq J \quad \mathbf{h}_0^{e,f} = \mathbf{0}$$

$$\mathbf{h}_j^{e,b} = \mathbf{F}_e(\mathbf{x}_j, \mathbf{h}_{j+1}^{e,b}) \quad J \geq j \geq 1 \quad \mathbf{h}_{J+1}^{e,b} = \mathbf{0}$$

$$\mathbf{h}_j^e = [\mathbf{h}_j^{e,f}; \mathbf{h}_j^{e,b}] \quad 1 \leq j \leq J$$

- The RNN-based decoder with attention model:

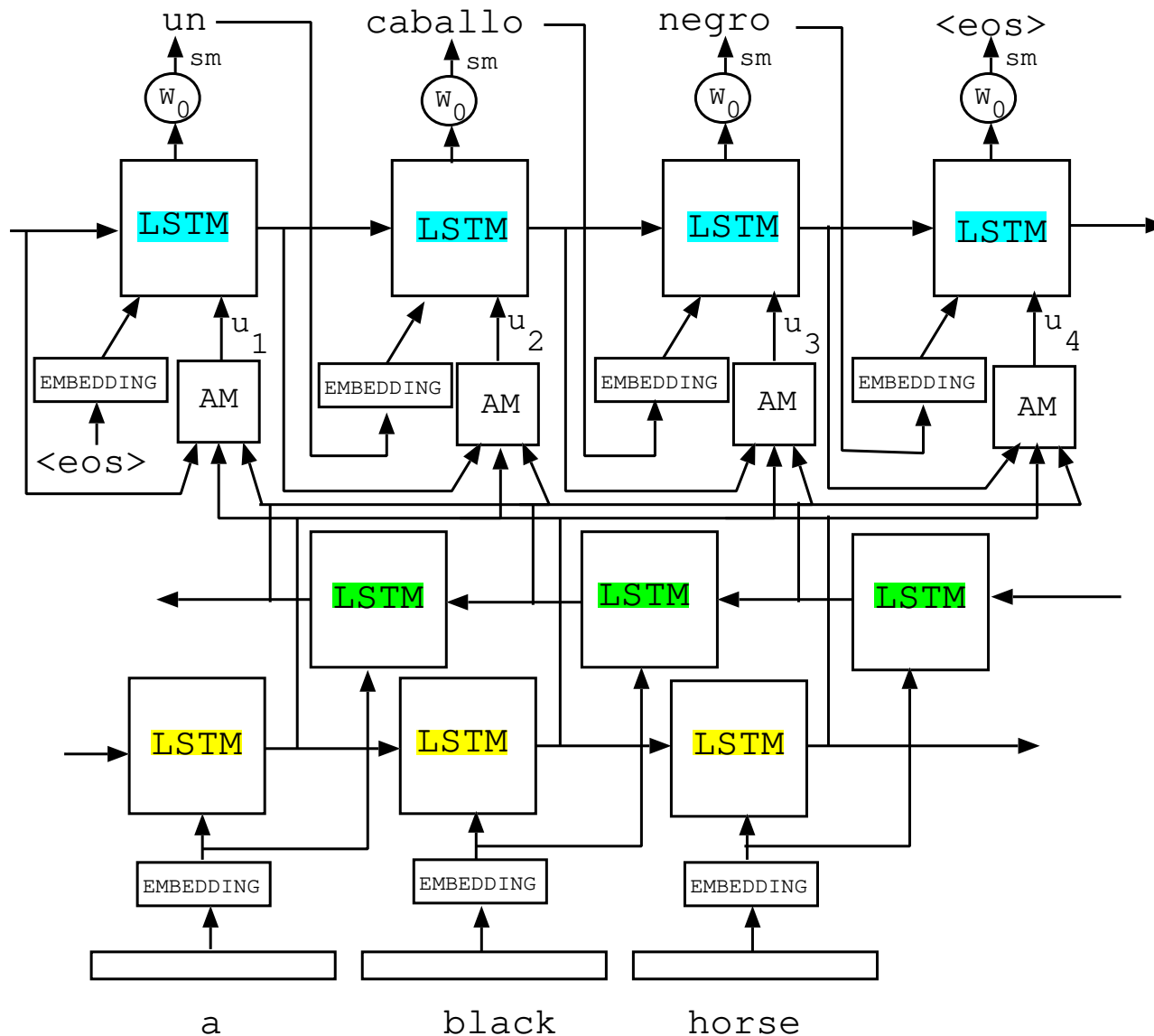
$$\mathbf{u}_i = \mathbf{a}(\mathbf{h}_1^e, \dots, \mathbf{h}_j^e, \mathbf{h}_{i-1}^d) \quad 1 \leq i \leq I$$

$$\mathbf{h}_i^d = \mathbf{F}_d(\mathbf{y}_{i-1}, \mathbf{h}_{i-1}^d, \mathbf{u}_i) \quad 1 \leq i \leq I \quad \mathbf{h}_0^d = \mathbf{0}$$

- Output generation:

$$p(y_1^I \mid x_1^J) = \prod_{i=1}^I p(y_i \mid y_1^{i-1}, u(x_1^J)) = \prod_{i=1}^I \mathbf{f}_{sm}(\mathbf{W}_O \mathbf{h}_i^d)_{i(y_i)}$$

Text translation with attention mechanism (Luong 2015)



Experiments: WMT14 En-Ge (Luong 2015)

Systems	BLEU
Combination of 8 models	23.0
Best system WMT14	20.7

Experiments in PRHLT: (Peris, 2017)

Task	Pair	TER (%)	
		PB	NMT
Xerox	Es-En	32.5	30.0
EU	Es-En	41.3	45.2

Other encoder-decoder architectures

- Multilayer encoder / multilayer decoder
- Conditional recurrent units (RNN blocks with attention models in between)
- GRUs

Index

- 1 Introduction ▷ 2
- 2 Synchronized feedforward neural networks ▷ 11
- 3 Synchronized recurrent neural networks ▷ 14
- 4 Neural machine translation ▷ 23
- 5 Attention models and recurrent neural networks ▷ 31
- 6 *Attention models for all* ▷ 40
- 7 Training and Decoding with NMT ▷ 59
- 8 Translation results with neural machine translation ▷ 67
- 9 Other issues of NMT ▷ 73
- 10 Bibliography ▷ 84

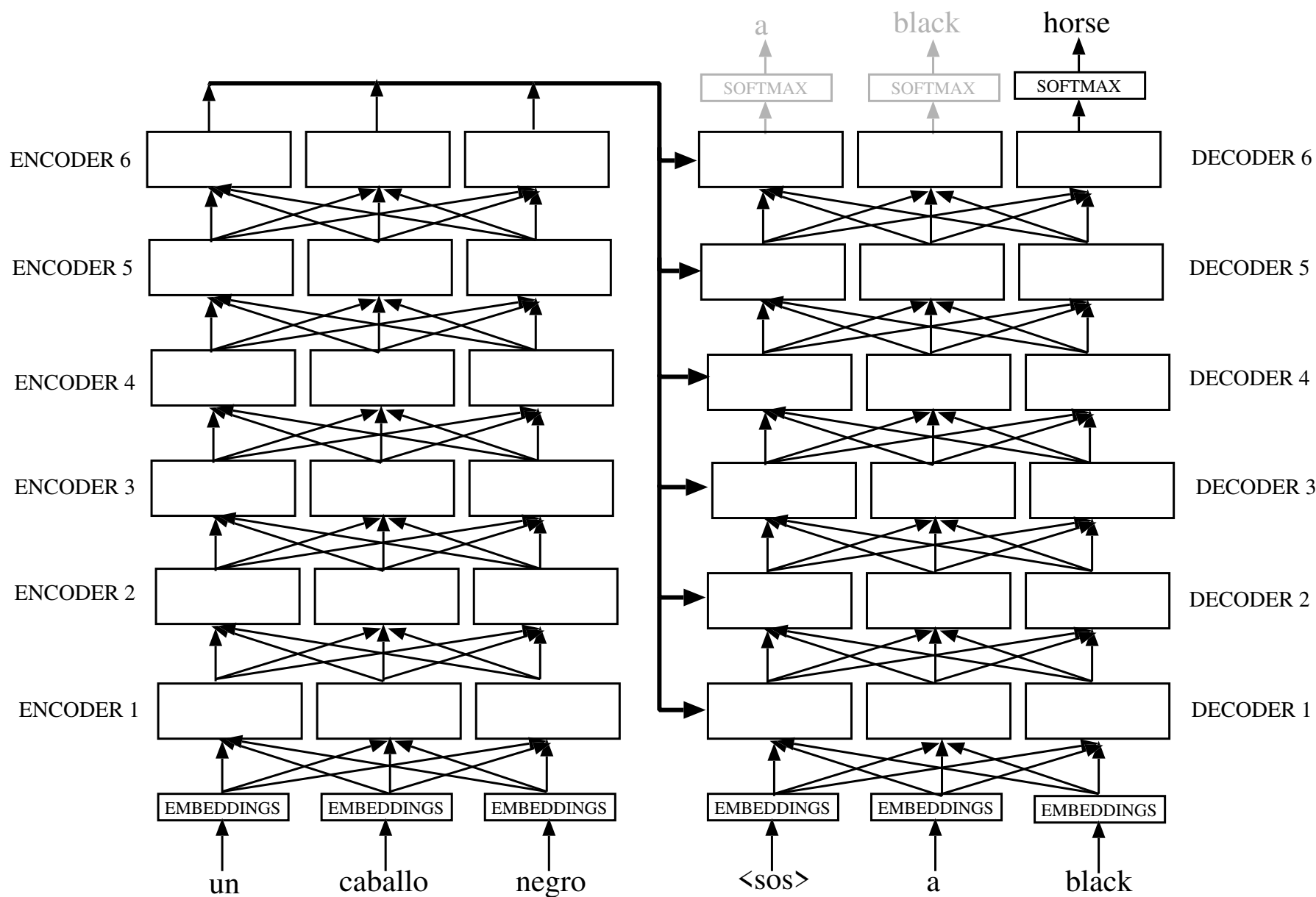
Transformer (Vaswani 2017)(Ney 2018)

- Goal: Given a source sentence x_1^J and a target sentence y_1^I , compute:

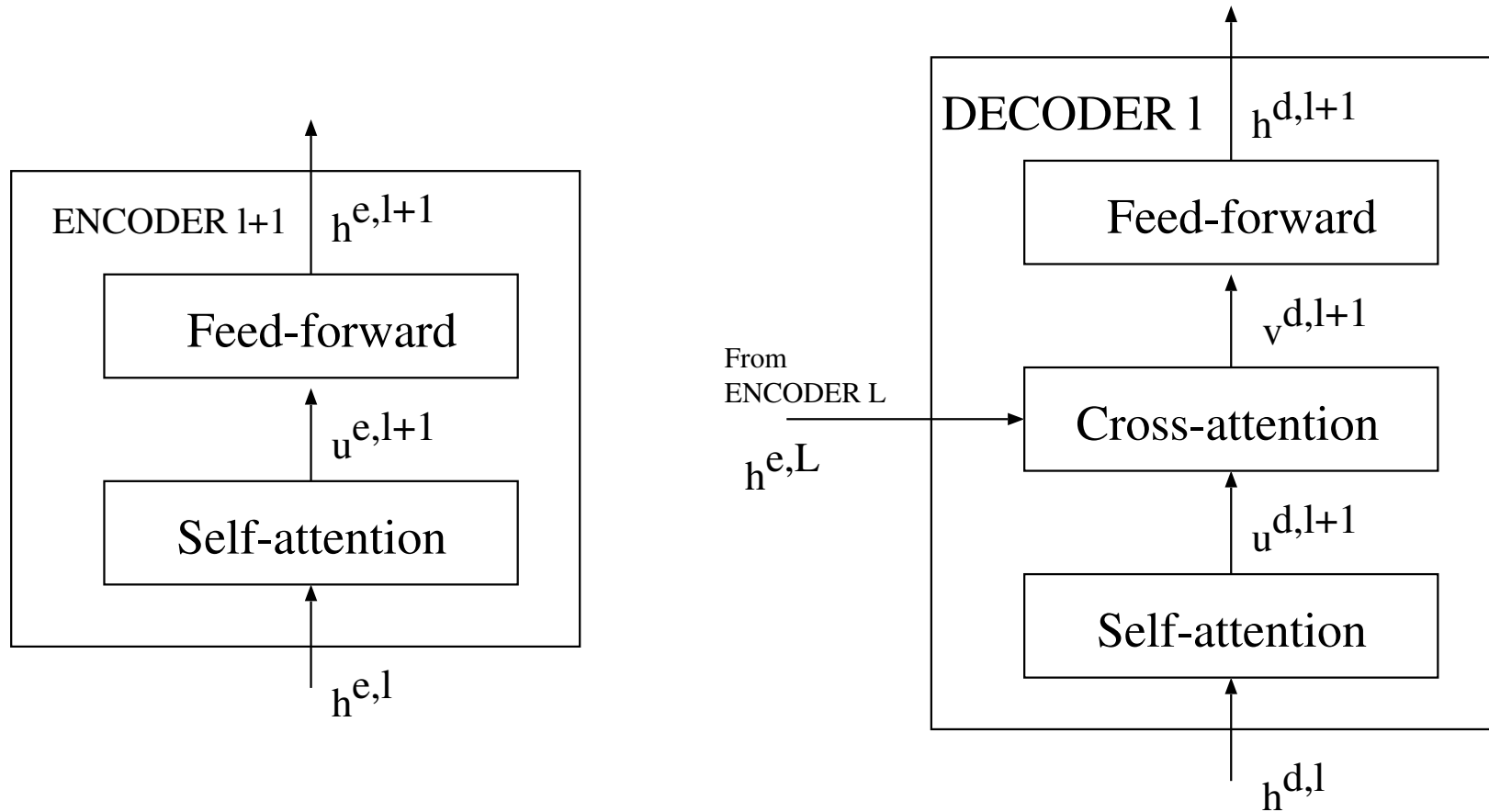
$$p(y_1^I \mid x_1^J) = \prod_{i=1}^I p(y_i \mid y_1^{i-1}, u(x_1^J))$$

- Feed-forward networks.
- Self-attention or intra-sentence attention: (j, j') & (i, i') in addition to the cross-attention (i, j) .
- Position encoding.
- Multi-head attention.
- Faster than RNN.

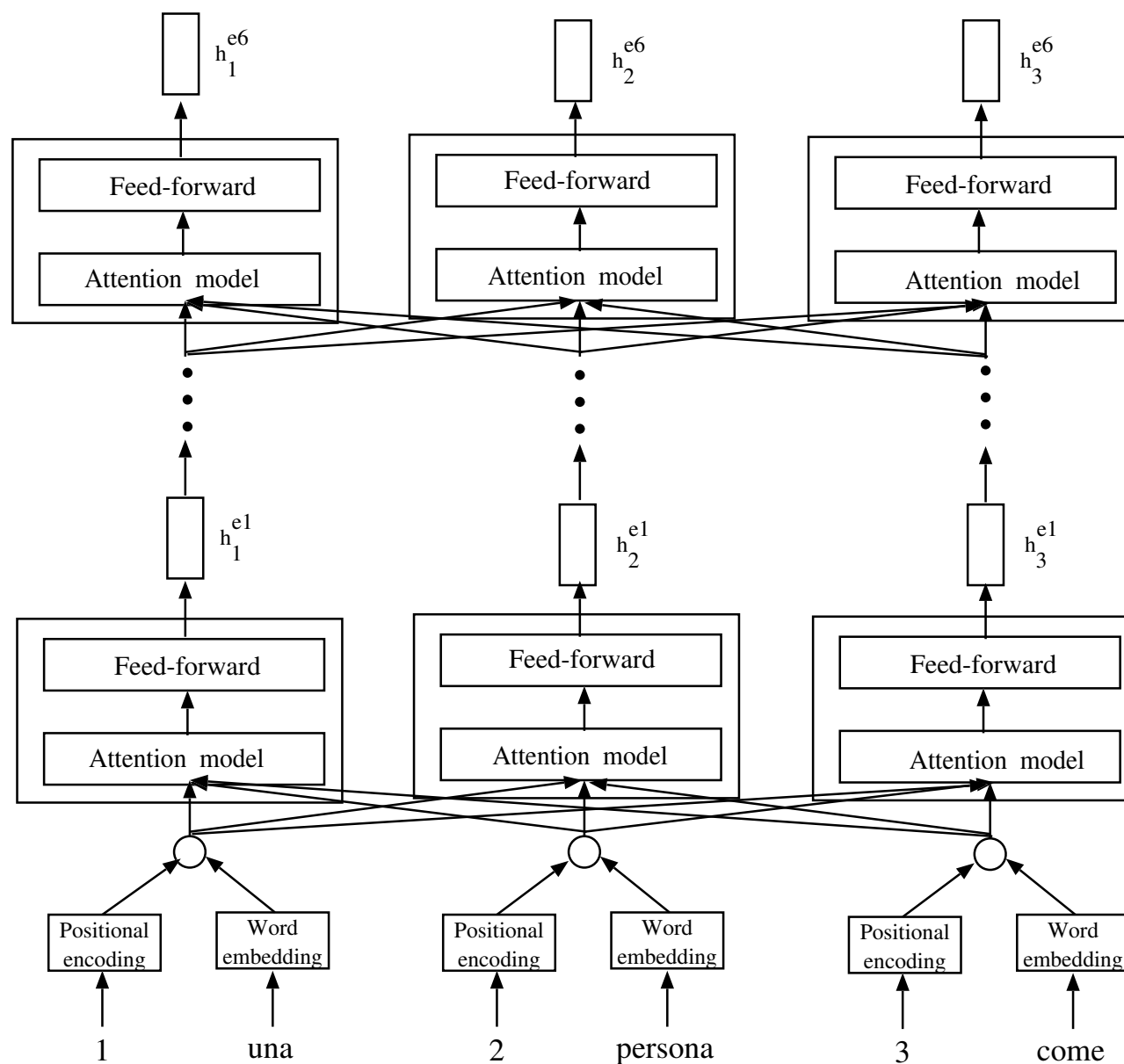
Transformer



Transformer



A simplified view of the encoder in the Transformer



Positional and word encodings

- A word x from a ordered vocabulary V_X with index $i(x)$
- **Word embeddings:** $\mathbf{W}_E(x) \equiv [\mathbf{W}_E]_{i(x)} = \mathbf{x} \in \mathbb{R}^{D_W}$: a row of $\mathbf{W}_E$ is the word embedding of x
- **Positional embeddings:** Given a sentence x_1^J the positional embedding of position j , $1 \leq j \leq J$ is $\mathbf{p}_j \in \mathbb{R}^{D_P}$:

$$\begin{aligned} p_{j,2k} &= \sin(j/10000^{2k/D_P}) \\ p_{j,2k+1} &= \cos(j/10000^{2k/D_P}) \end{aligned}$$

- **Relative positional embeddings.**
- Learned positional embeddings.

Self-attention models for the encoder

Given a sequence of encoder states $\mathbf{h}_j^{e,l}$ for $1 \leq j \leq J$ in layer $1 \leq l \leq L$, a **self-attention model** for a position j in a layer $l + 1$ is a function:

$$\mathbf{u}_j^{e,l+1} = \mathbf{a}(\mathbf{h}_1^{e,l}, \dots, \mathbf{h}_J^{e,l}, \mathbf{h}_j^{e,l}) \quad 1 \leq i \leq J$$

where $\mathbf{a}$ is:

$$\mathbf{u}_j^{e,l+1} = \sum_{j'=1}^J f_{j'}(\mathbf{h}_1^{e,l}, \dots, \mathbf{h}_J^{e,l}, \mathbf{h}_j^{e,l}) \mathbf{W}_V^e \mathbf{h}_{j'}^{e,l} \quad 1 \leq i \leq J$$

$$f_{j'}(\mathbf{h}_1^{e,l}, \dots, \mathbf{h}_J^{e,l}, \mathbf{h}_j^{e,l}) = \frac{\exp(\mathbf{W}_Q^e \mathbf{h}_j^{e,l} \cdot \mathbf{W}_K^e \mathbf{h}_{j'}^{e,l})}{\sum_{j''=1}^J \exp(\mathbf{W}_Q^e \mathbf{h}_j^{e,l} \cdot \mathbf{W}_K^e \mathbf{h}_{j''}^{e,l})} = p(j' | j) \quad 1 \leq j, j' \leq J$$

The encoder of Transformer

Given a source sentence x_1^J ,

- Initialization: $\mathbf{h}_j^{e,0} = \mathbf{x}_j = \mathcal{E}(x_j) \quad 1 \leq j \leq J$

- In layer l of the encoder ($1 \leq l < L$)

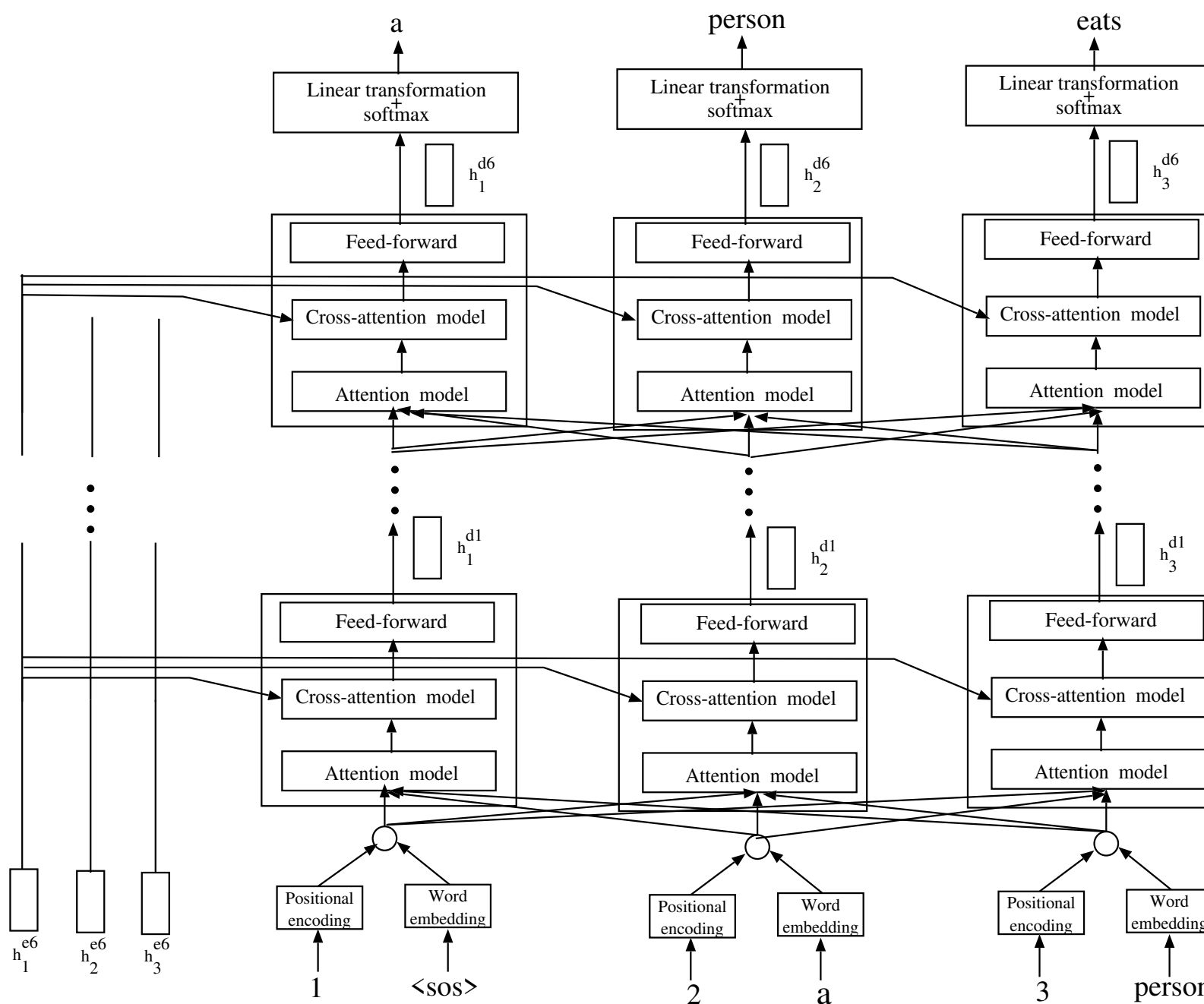
 - Self-attention model:

$$\mathbf{u}_j^{e,l+1} = \mathbf{a}(\mathbf{h}_1^{e,l}, \dots, \mathbf{h}_J^{e,l}, \mathbf{h}_j^{e,l}) \quad 1 \leq i \leq J$$

 - Feed-forward network:

$$\mathbf{h}_j^{e,l+1} = \mathbf{F}_f(\mathbf{u}_j^{e,l+1}) \quad 1 \leq j \leq J$$

A simplified view of the decoder in the Transformer



Self-attention models for the decoder

Given a sequence of decoder states $\mathbf{h}_{i'}^{d,l}$ for $1 \leq i' \leq i$ for some i in layer $1 \leq l \leq L$, a **self-attention model** for a position $i' \leq i$ in a layer $l + 1$ is a function:

$$\mathbf{u}_{i'}^{d,l+1} = \mathbf{a}(\mathbf{h}_1^{d,l}, \dots, \mathbf{h}_{i'}^{d,l}, \mathbf{h}_{i'}^{d,l}) \quad 1 \leq i' \leq i$$

where $\mathbf{a}$ is:

$$\mathbf{u}_{i'}^{d,l+1} = \sum_{i''=1}^i f_{i''}(\mathbf{h}_1^{d,l}, \dots, \mathbf{h}_{i'}^{d,l}, \mathbf{h}_{i'}^{d,l}) \mathbf{W}_V^d \mathbf{h}_{i''}^{d,l}$$

$$f_{i''}(\mathbf{h}_1^{d,l}, \dots, \mathbf{h}_{i'}^{d,l}, \mathbf{h}_{i'}^{d,l}) = \frac{\exp(\mathbf{W}_Q^d \mathbf{h}_{i'}^{d,l} \cdot \mathbf{W}_K^d \mathbf{h}_{i''}^{d,l})}{\sum_{j'''=1}^i \exp(\mathbf{W}_Q^d \mathbf{h}_{i'}^{d,l} \cdot \mathbf{W}_K^d \mathbf{h}_{j'''}^{d,l})} = p(i'' \mid i') \quad 1 \leq i', i'' \leq i$$

Cross-attention models for the decoder

Given a sequence of output of self-attentions in the decoder $\mathbf{u}_{i'}^{d,l}$ for $1 \leq i' \leq i$ and some i in layer $1 \leq l \leq L$, a **cross-attention model** for a position $i' \leq i$ in a layer $l + 1$ is a function:

$$\mathbf{v}_{i'}^{d,l+1} = \mathbf{a}(\mathbf{h}_1^{e,L}, \dots, \mathbf{h}_J^{e,L}, \mathbf{u}_{i'}^{d,l+1}) \quad 1 \leq i' \leq i$$

where $\mathbf{a}_c$ is in:

$$\mathbf{v}_{i'}^{d,l+1} = \sum_{j=1}^J f_j(\mathbf{h}_1^{e,L}, \dots, \mathbf{h}_J^{e,L}, \mathbf{u}_{i'}^{d,l+1}) \mathbf{W}_V^c \mathbf{h}_j^{e,L}$$

$$f_j(\mathbf{h}_1^{e,L}, \dots, \mathbf{h}_J^{e,L}, \mathbf{u}_{i'}^{d,l+1}) = \frac{\exp(\mathbf{W}_Q^c \mathbf{u}_{i'}^{d,l+1} \cdot \mathbf{W}_K^c \mathbf{h}_j^{e,L})}{\sum_{j=1}^J \exp(\mathbf{W}_Q^c \mathbf{u}_{i'}^{d,l+1} \cdot \mathbf{W}_K^c \mathbf{h}_j^{e,L})} = p(j \mid i'); 1 \leq i' \leq i, \quad 1 \leq j \leq J$$

The decoder of Transformer

Given a sequence of states of the encoder $\mathbf{h}_j^{e,L}$ $1 \leq j \leq J$,

- For $1 \leq i \leq I$

- Initialization: $\mathbf{u}_{i-1}^{d,1} = \mathbf{y}_{i-1} = \mathcal{E}(y_{i-1})$, ($y_0 = "$ < *sos* > ")

- In layer l of the decoder ($1 \leq l < L$):

- * Self-attention model:

$$\mathbf{u}_i^{d,l+1} = \mathbf{a}(\mathbf{h}_1^{d,l}, \dots, \mathbf{h}_i^{d,l}, \mathbf{h}_i^{d,l})$$

- * Cross-attention model:

$$\mathbf{v}_i^{d,l+1} = \mathbf{a}(\mathbf{h}_1^{e,L}, \dots, \mathbf{h}_J^{e,L}, \mathbf{u}_i^{d,l+1})$$

- * Feed-forward network:

$$\mathbf{h}_i^{d,l+1} = \mathbf{F}_f(\mathbf{v}_i^{d,l+1})$$

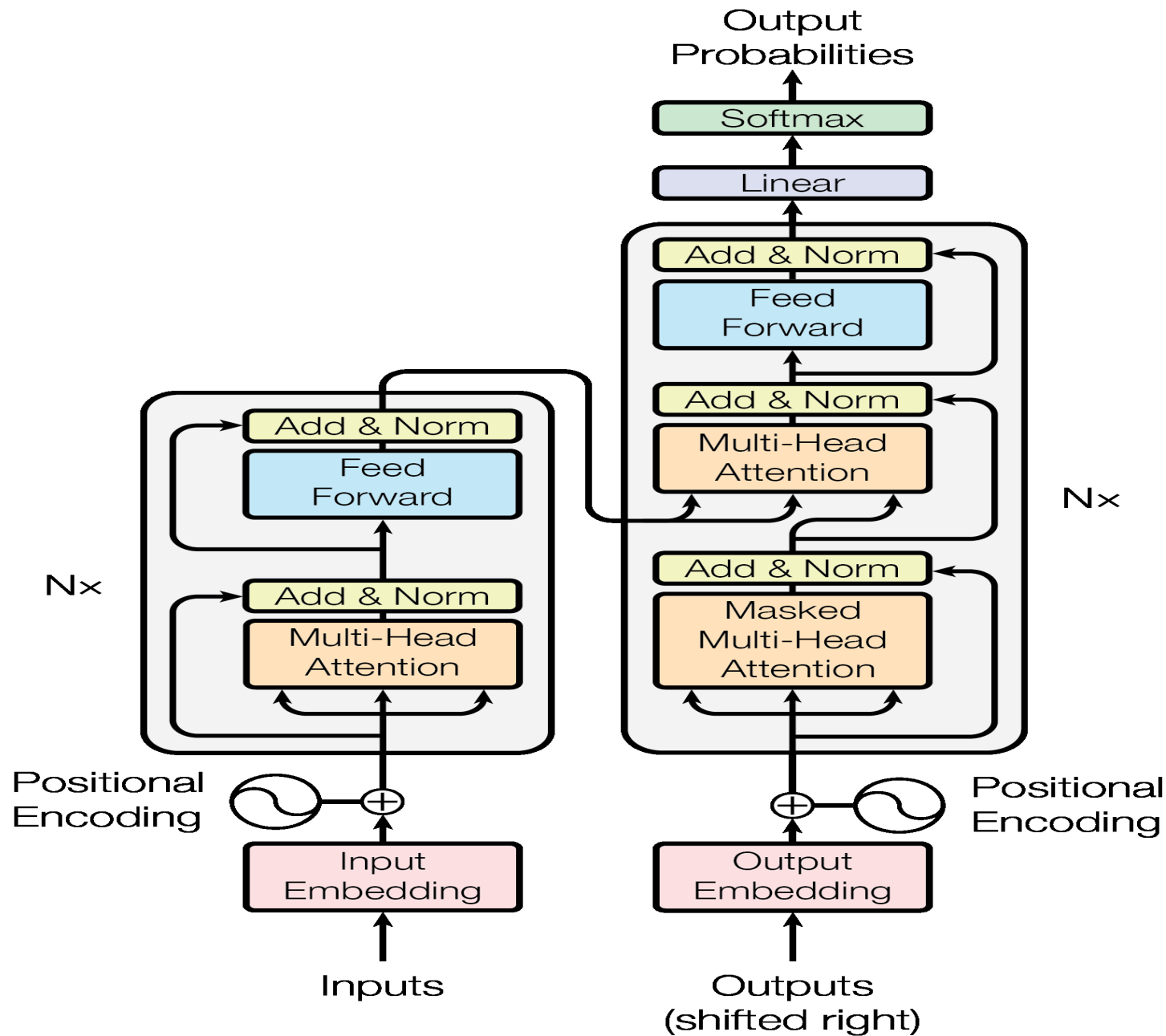
The decoder of Transformer (Vaswani 2017)

Given the final states of the decoder $\mathbf{u}_i^L$ $1 \leq i \leq I$,

- Output generation:

$$p(y_1^I \mid x_1^J) = \prod_{i=1}^I p(y_i \mid y_1^{i-1}, u(x_1^J)) = \prod_{i=1}^I \mathbf{f}_{sm}(\mathbf{W} \mathbf{h}_i^L)_{i(y_i)}$$

Transformer (Vaswani 2017)



Some details and additional features of Transformer

- Residual networks
- Layer normalization
- Feed-forward networks
- Multi-head attention

Some details and additional features of Transformer (Xiong arXiv 2020)

- Given $\mathbf{z} \in \mathbb{R}^d$ and a function $\mathbf{f} : \mathbb{R}^d \rightarrow \mathbb{R}^d$, a **residual function** $\mathbf{R}$ is defined as:

$$\mathbf{R}(\mathbf{z}, \mathbf{f}(\mathbf{z})) = \mathbf{f}(\mathbf{z}) + \mathbf{z}$$

- Given a sequence of vectors $\mathbf{z}_1, \dots, \mathbf{z}_K$ from a layer with $\mathbf{z}_k \in \mathbb{R}^d$ for $1 \leq k \leq K$, a **layer normalization** $\mathbf{N}$ is $\mathbf{N}(\mathbf{z}_1, \dots, \mathbf{z}_K) = (\bar{\mathbf{z}}_1, \dots, \bar{\mathbf{z}}_K)$ such that:

$$\bar{z}_{k,i} = \gamma \frac{z_{k,i} - \mu_i}{\sigma} + \beta \quad 1 \leq k \leq K, 1 \leq i \leq d$$

where γ and β are hyper-parameters, $\mu_i = \frac{\sum_{k=1}^K z_{k,i}}{K}$ and $\sigma_i = \frac{\sum_{k=1}^K (z_{k,i} - \mu_i)^2}{K}$ $1 \leq i \leq d$

- Feed-forward networks, $\mathbf{F}_f$: Given a $\mathbf{z} \in \mathbb{R}^d$.

$$\mathbf{F}_f(\mathbf{z}) = \mathbf{W}_2 \mathbf{f}_{ReLU}(\mathbf{W}_1 \mathbf{z} + \mathbf{b}_1) + \mathbf{b}_2$$

where $\mathbf{W}_2$, $\mathbf{W}_1$ are the weights of the second and first layer of the feed-forward networks, and $\mathbf{b}_2$ and $\mathbf{b}_1$ the corresponding bias.

Multi-head attention in Transformer

For N heads and $1 \leq l \leq L$, $1 \leq j \leq J$ and $1 \leq i \leq I$:

- Encoder self-attention:

$$\begin{aligned} \mathbf{u}_{j,n}^{e,l+1} &= \mathbf{a}_n(\mathbf{h}_1^{e,l}, \dots, \mathbf{h}_J^{e,l}, \mathbf{h}_j^{e,l}) \quad 1 \leq j \leq J \text{ and } 1 \leq n \leq N \\ &= \sum_{j'=1}^J \frac{\exp(\mathbf{W}_Q^{e,n} \mathbf{h}_j^{e,l} \cdot \mathbf{W}_K^{e,n} \mathbf{h}_{j'}^{e,l})}{\sum_{j''=1}^J \exp(\mathbf{W}_Q^{e,n} \mathbf{h}_j^{e,l} \cdot \mathbf{W}_K^{e,n} \mathbf{h}_{j''}^{e,l})} \mathbf{W}_V^{e,n} \mathbf{h}_{j'}^{e,l} \quad 1 \leq j, j' \leq J \end{aligned}$$

$\mathbf{W}_Q^{e,n}, \mathbf{W}_K^{e,n}, \mathbf{W}_V^{e,n}$ are matrix of $D_W/N \times D_W$

$$\mathbf{u}_j^{e,l+1} = \left[\mathbf{u}_{j,1}^{e,l+1}, \dots, \mathbf{u}_{j,N}^{e,l+1} \right]; \quad \mathbf{h}_j^{e,l+1} = \mathbf{F}_f(\mathbf{u}_j^{e,l+1}) \quad 1 \leq j \leq J$$

Multi-head attention in Transformer

For N heads and $1 \leq l \leq L$, $1 \leq j \leq J$ and $1 \leq i \leq I$:

- Decoder self-attention:

$$\begin{aligned} \mathbf{u}_{i,n}^{d,l+1} &= \mathbf{a}_n(\mathbf{h}_1^{d,l}, \dots, \mathbf{h}_J^{d,l}, \mathbf{h}_i^{d,l}) \quad 1 \leq i \leq I \text{ and } 1 \leq n \leq N \\ &= \sum_{i'=1}^I \frac{\exp(\mathbf{W}_Q^{d,n} \mathbf{h}_i^{d,l} \cdot \mathbf{W}_K^{d,n} \mathbf{h}_{i'}^{d,l})}{\sum_{i''=1}^I \exp(\mathbf{W}_Q^{d,n} \mathbf{h}_i^{d,l} \cdot \mathbf{W}_K^{d,n} \mathbf{h}_{i''}^{d,l})} \mathbf{W}_V^{d,n} \mathbf{h}_{i'}^{d,l} \quad 1 \leq i, i' \leq I \end{aligned}$$

$\mathbf{W}_Q^{d,n}, \mathbf{W}_K^{d,n}, \mathbf{W}_V^{d,n}$ are matrix of $D_W/N \times D_W$

$$\mathbf{u}_i^{d,l+1} = \left[\mathbf{u}_{i,1}^{d,l+1}, \dots, \mathbf{u}_{i,N}^{d,l+1} \right]; \quad \mathbf{h}_i^{d,l+1} = \mathbf{F}_f(\mathbf{u}_i^{d,l+1}) \quad 1 \leq i \leq I$$

Multi-head attention in Transformer

For N heads and $1 \leq l \leq L$, $1 \leq j \leq J$ and $1 \leq i \leq I$:

- Decoder cross-attention:

$$\begin{aligned} \mathbf{v}_{i,n}^{d,l+1} &= \mathbf{a}_n(\mathbf{h}_1^{e,L}, \dots, \mathbf{h}_J^{e,L}, \mathbf{u}_i^{d,l+1}) \quad 1 \leq i \leq I \text{ and } 1 \leq n \leq N \\ &= \sum_{j=1}^J \frac{\exp(\mathbf{W}_Q^{c,n} \mathbf{u}_i^{d,l+1} \cdot \mathbf{W}_K^{c,n} \mathbf{h}_j^{e,L})}{\sum_{j=1}^J \exp(\mathbf{W}_Q^{c,n} \mathbf{u}_i^{d,l+1} \cdot \mathbf{W}_K^{c,n} \mathbf{h}_j^{e,L})} \mathbf{W}_V^{c,n} \mathbf{h}_j^{e,L} \quad 1 \leq i \leq I \end{aligned}$$

$\mathbf{W}_Q^{c,n}, \mathbf{W}_K^{c,n}, \mathbf{W}_V^{c,n}$ are matrix of $D_W/N \times D_W$

$$\mathbf{v}_i^{d,l+1} = \left[\mathbf{v}_{i,1}^{d,l+1}, \dots, \mathbf{v}_{i,N}^{d,l+1} \right]; \quad \mathbf{h}_i^{d,l+1} = \mathbf{F}_f(\mathbf{v}_i^{d,l+1}) \quad 1 \leq i \leq I$$

Index

- 1 Introduction ▷ 2
- 2 Synchronized feedforward neural networks ▷ 11
- 3 Synchronized recurrent neural networks ▷ 14
- 4 Neural machine translation ▷ 23
- 5 Attention models and recurrent neural networks ▷ 31
- 6 Attention models for all ▷ 40
- 7 *Training and Decoding with NMT* ▷ 59
- 8 Translation results with neural machine translation ▷ 67
- 9 Other issues of NMT ▷ 73
- 10 Bibliography ▷ 84

Training with NMT (Koehn, 2017)

- Given a training set of bilingual pairs T , maximize the log-likelihood:

$$\begin{aligned}
 \widehat{\mathbf{W}} &= \operatorname{argmax}_{\mathbf{W}} \mathcal{F}_T(\mathbf{W}) \equiv \operatorname{argmax}_{\mathbf{W}} \sum_{(x_1^J, y_1^I) \in T} \log p(y_1^I \mid x_1^J; \mathbf{W}) \\
 &= \operatorname{argmax}_{\mathbf{W}} \sum_{(x_1^J, y_1^I) \in T} \sum_{i=1}^I \log p(y_i \mid y_1^{i-1}, x_1^J; \mathbf{W}) \\
 &= \operatorname{argmax}_{\mathbf{W}} \sum_{(x_1^J, y_1^I) \in T} \sum_{i=1}^I \log \mathbf{f}_{sm}(\mathbf{W}_O \mathbf{h}_i^d(x_1^J))_{i(y_i)}
 \end{aligned}$$

- Optimization algorithms: Based on backpropagation through time using stochastic gradient descent.
 - Shuffle the training corpus
 - Break up the corpus into maxi-batches
 - Break up each maxi-batch into mini-batches
 - Compute gradients for each mini-batches
 - Update all the parameters (weights and word embeddings) at the end of a maxi-batch

Gradient descent optimization algorithms (Ruder 2016)

- **SDG** (stochastic gradient descent).
 - **SGD with momentum**
 - **Adagrad** (Adaptive Gradient)
 - **Adadelta** (an extension of Adagrad)
 - **Adam** (Adaptive Moment Estimation)
 - NAG, RMSProp, AdaMax, Nadam, ...
-
- Implementation through computational graphs in TensorFlow, PyTorch, JAX (<https://jax.readthedocs.io/en/latest/>), ...

Training issues

- Early stopping.
- Batch and layer normalization.
- Data augmentation.
- Hyperparameters
 - Dimension of source word embeddings.
 - Dimension of target word embeddings.
 - Number of LSTM/GRUs in encoder.
 - Number of LSTM/GRUs in decoder.
 - Number of layers in encoder of Transformer.
 - Number of layers in decoder of Transformer.
 - Initial learning rate.
 - Patience.
 - Dropout parameter.
 - Weight decay.
 - Noisy injection (to inputs, outputs, ...)
 - ...

Inference (decoding) with NMT (Koehn, 2017)

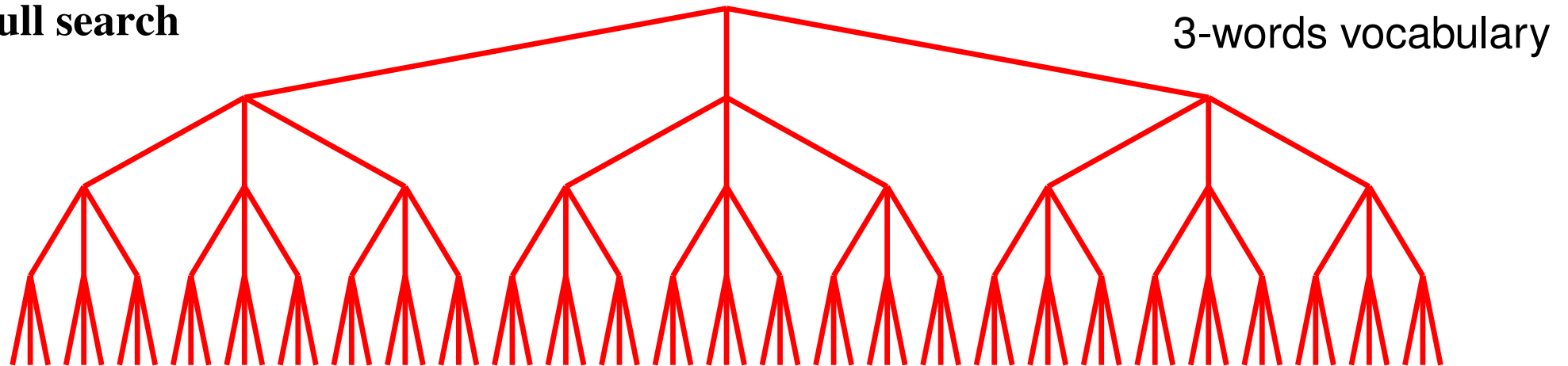
- For a source sentence x_1^J search for a target sentence $\hat{y}_1^{\hat{I}}$,

$$\begin{aligned}\hat{y}_1^{\hat{I}} &= \operatorname{argmax}_{I, y_1^I} p(y_1^I \mid x_1^J; \mathbf{W}) \\ &= \operatorname{argmax}_{I, y_1^I} \prod_{i=1}^I p(y_i \mid y_1^{i-1}, u(x_1^J); \mathbf{W}) \\ &= \operatorname{argmax}_{I, y_1^I} \prod_{i=1}^I \mathbf{f}_{sm}(\mathbf{W}_O \mathbf{h}_i^d(x_1^J))_{i(y_i)}\end{aligned}$$

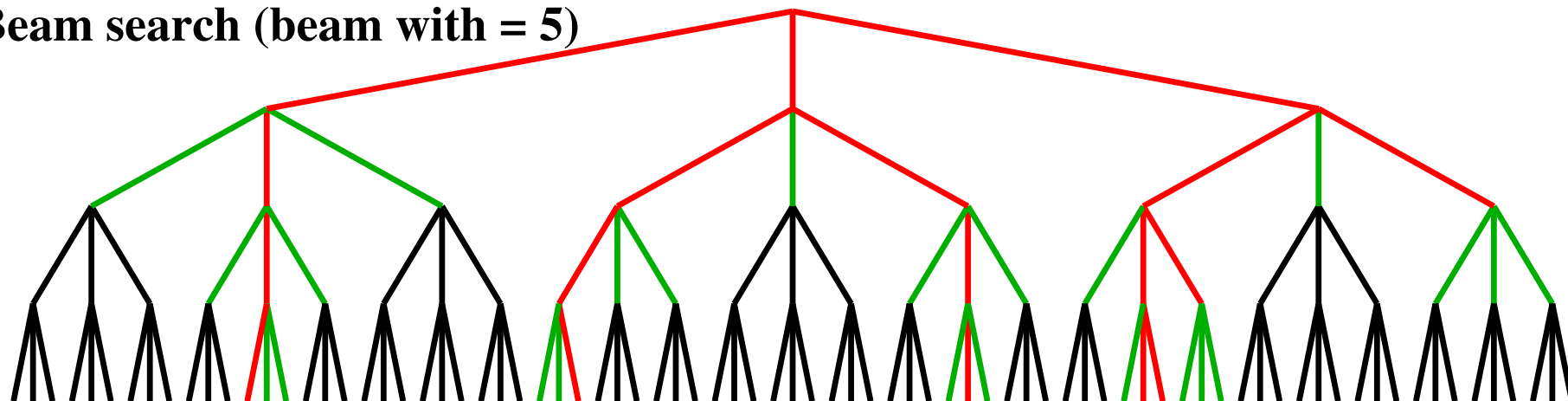
- Translation:
 - **Greedy** approach: for each i the maximum from the softmax output.
 - **Smart tree search** with **beam search** (suboptimal search)
 - * At the beginning of the process there is a empty list of target hypotheses.
 - * At each target position there is a bounded prefix tree of K target prefixes. For each prefix, a bounded set of extended prefixes is generated by appending the most promising words.
 - * The process ends when a “eof” is generated.

Decoding with NMT (Ney, 2018)

Full search



Beam search (beam with = 5)

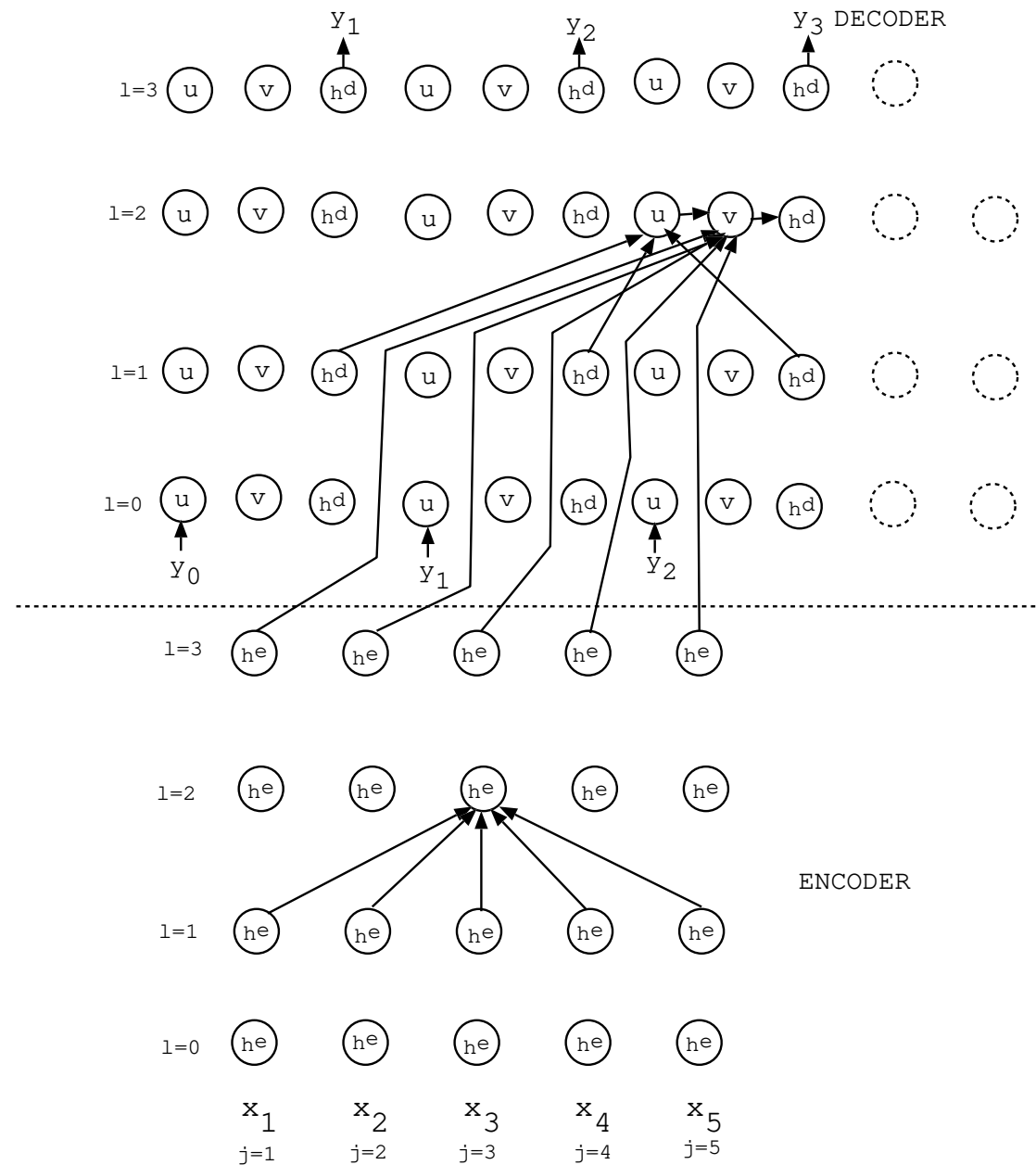


black=possible arcs

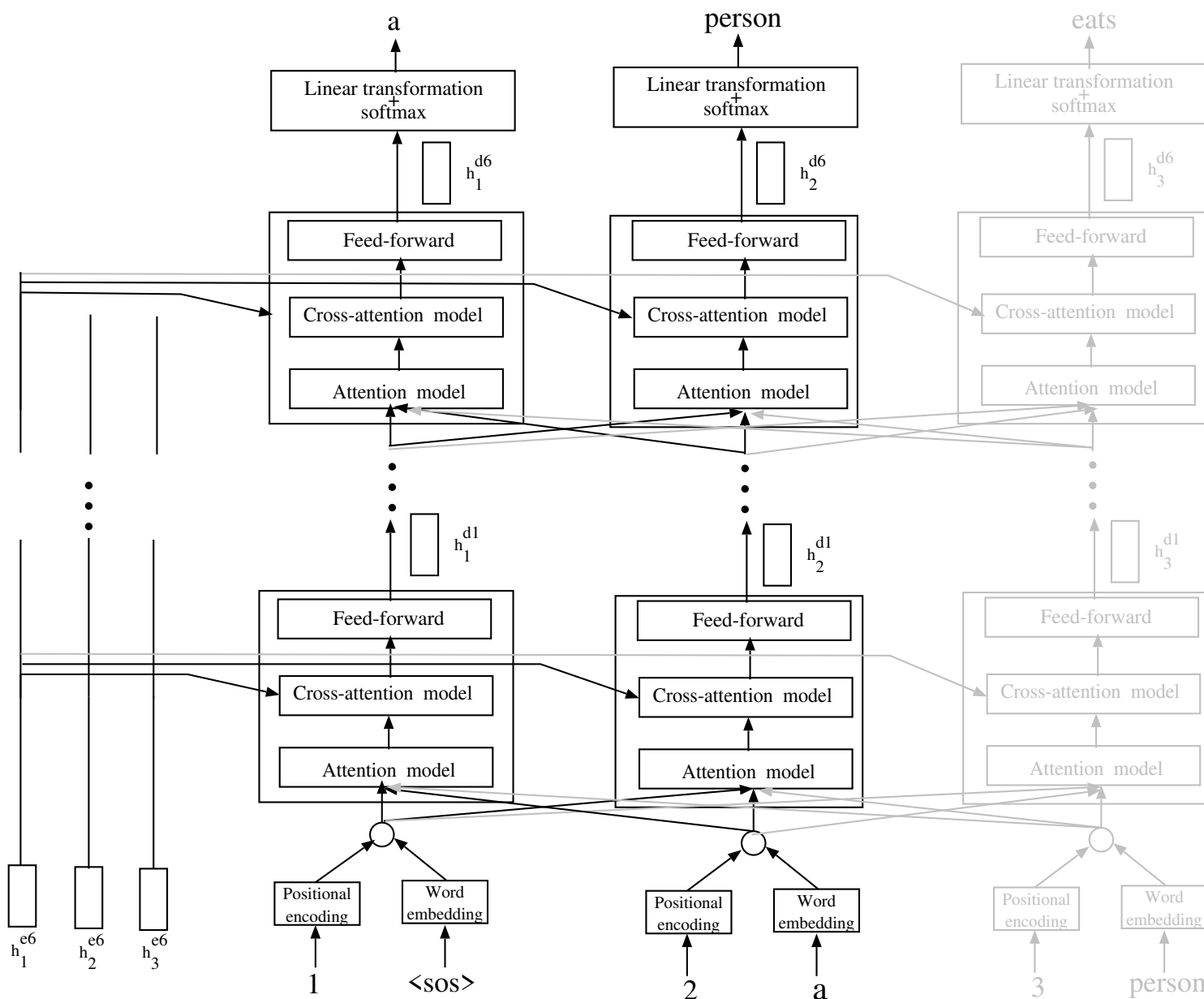
green = extended & discarded

red = extended & preserved

Text translation with Transformer (Ney 2018)



Text translation with Transformer with look-ahead masks



Index

- 1 Introduction ▷ 2
- 2 Synchronized feedforward neural networks ▷ 11
- 3 Synchronized recurrent neural networks ▷ 14
- 4 Neural machine translation ▷ 23
- 5 Attention models and recurrent neural networks ▷ 31
- 6 Attention models for all ▷ 40
- 7 Training and Decoding with NMT ▷ 59
- 8 *Translation results with neural machine translation* ▷ 67
- 9 Other issues of NMT ▷ 73
- 10 Bibliography ▷ 84

Encoder-decoder approach to NMT

Comparative results

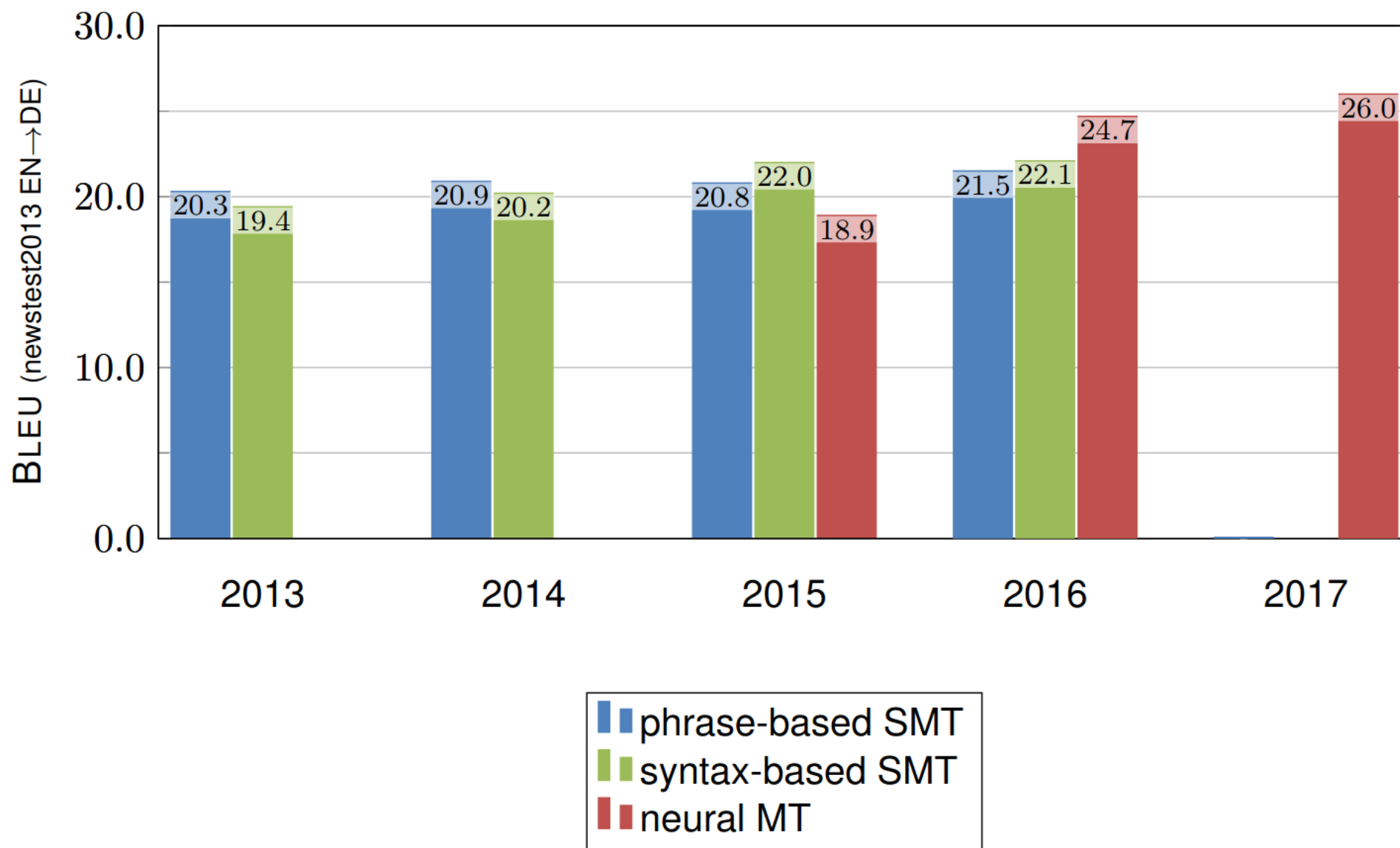
Corpus for text translation

- XRCE: Printer manuals.
- TED: Transcription of videos from expert speakers on education, business, science, tech and creativity.
<https://www.ted.com/talks>
- UFAL: Medical texts.
https://ufal.mff.cuni.cz/ufal_medical_corpus
- EUROPARL: Proceedings of the European Parliament
<https://www.statmt.org/europarl/>

Text translation results (Peris 2019)

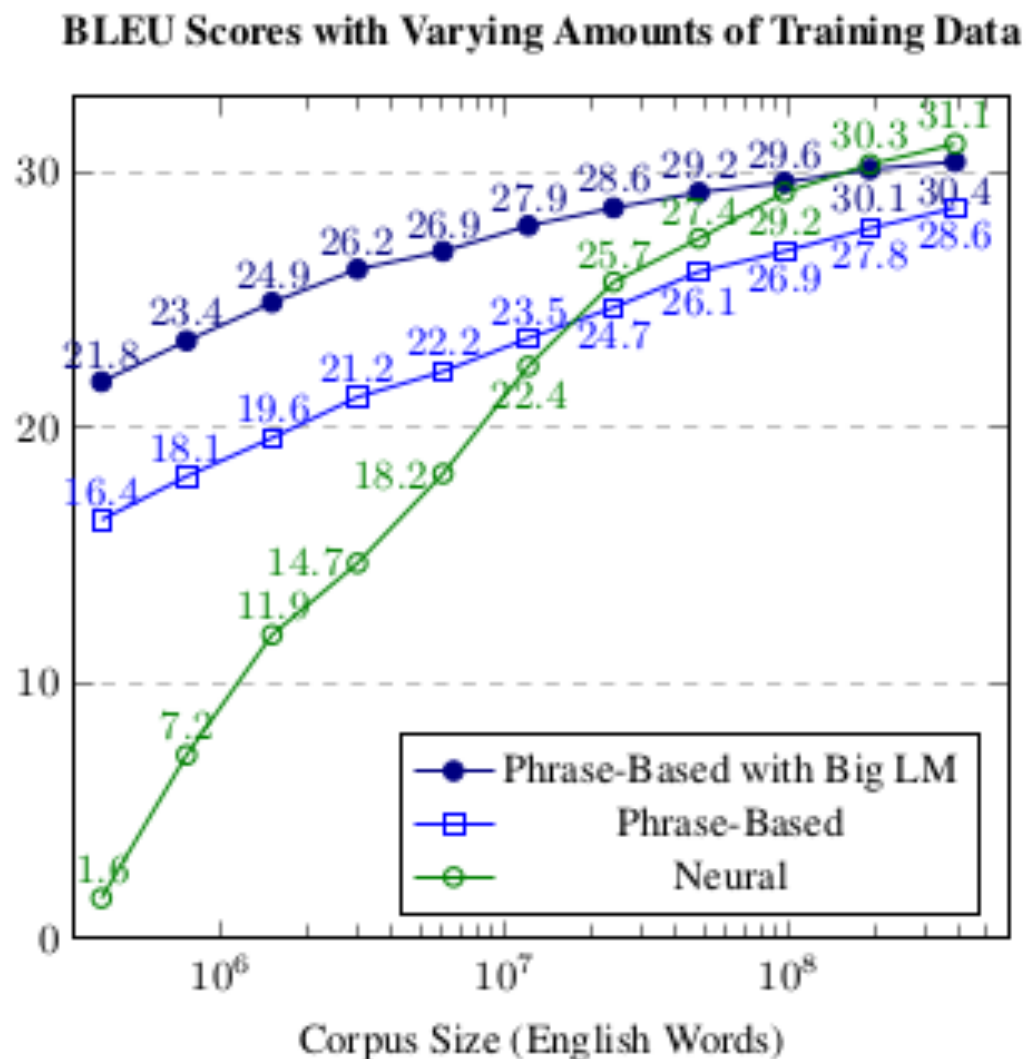
Task	Languages	Recurrent		Transformer		PB-SMT	
		BLEU	TER	BLEU	TER	BLEU	TER
XRCE	EN-FR	38.0	51.9	32.0	57.2	37.9	50.2
XRCE	GE-EN	36.2	51.1	31.3	54.9	36.8	50.1
TED	FR-EN	32.4	46.6	30.1	49.5	29.9	50.5
TED	ZH-EN	13.7	75.7	11.5	76.7	11.0	77.5
UFAL	EN-FR	37.2	46.1	37.8	45.9	35.0	47.9
UFAL	ES-EN	44.4	35.4	43.1	36.4	38.7	40.4
EUROPARL	EN-ES	25.0	58.6	26.1	55.4	24.6	56.8
EUROPARL	GE-EN	21.2	62.9	22.3	60.7	20.4	60.9

Edinburgh's WMT results over the years (Sennrich et al. 2017)



PBMT vs NMT and the size of training data (Koehn & Knowles. 2017)

English-Spanish



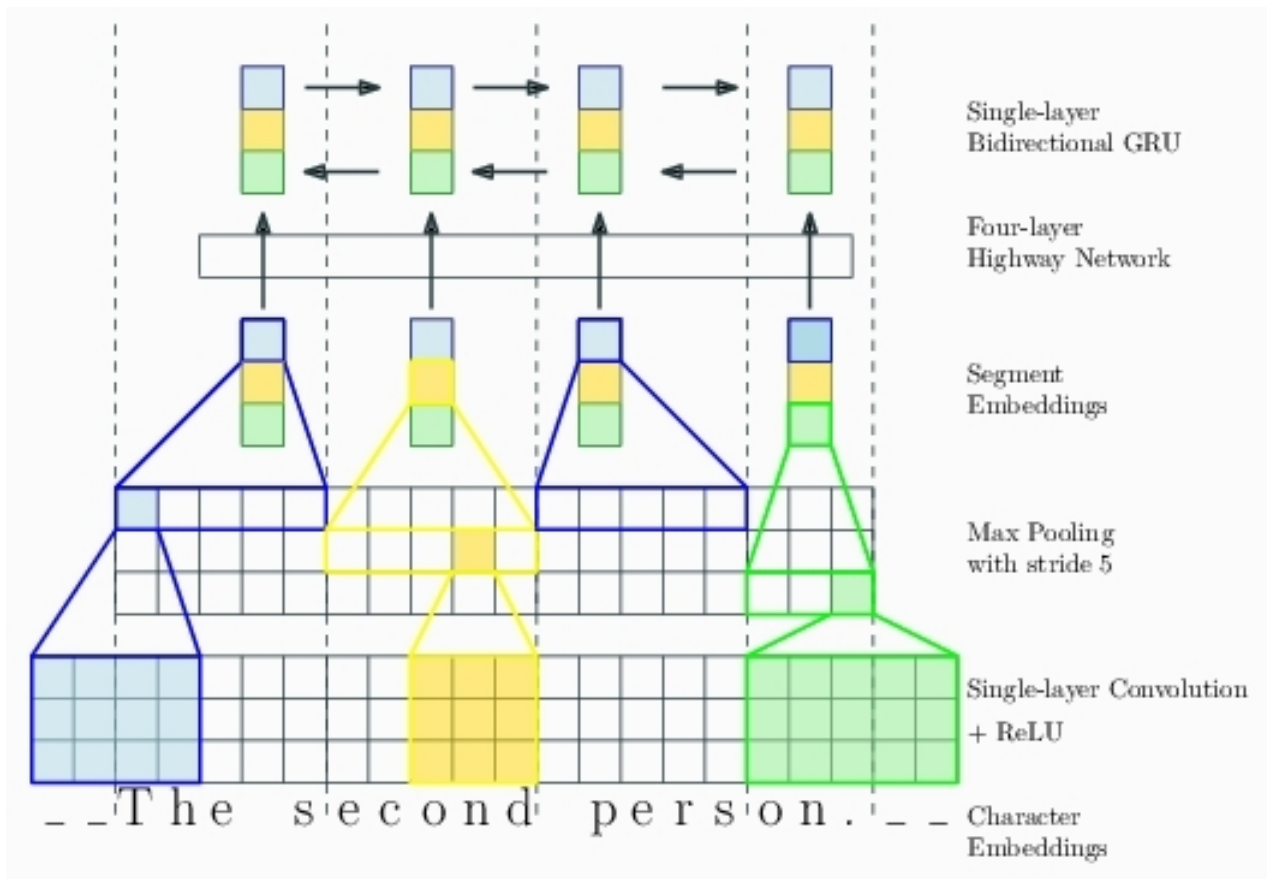
Index

- 1 Introduction ▷ 2
- 2 Synchronized feedforward neural networks ▷ 11
- 3 Synchronized recurrent neural networks ▷ 14
- 4 Neural machine translation ▷ 23
- 5 Attention models and recurrent neural networks ▷ 31
- 6 Attention models for all ▷ 40
- 7 Training and Decoding with NMT ▷ 59
- 8 Translation results with neural machine translation ▷ 67
- 9 *Other issues of NMT* ▷ 73
- 10 Bibliography ▷ 84

Character based text translation (Larriba 2018)

- Pros:
 - Smaller vocabularies.
 - Reduce out-of-vocabulary words.
 - Profit from in-word lexical information
- Cons:
 - Longer input sequences.
 - References at word level.
- Approaches:
 - Fully character based NMT
 - Encoder based on convolutional character NN
 - Word embedding from character embedding

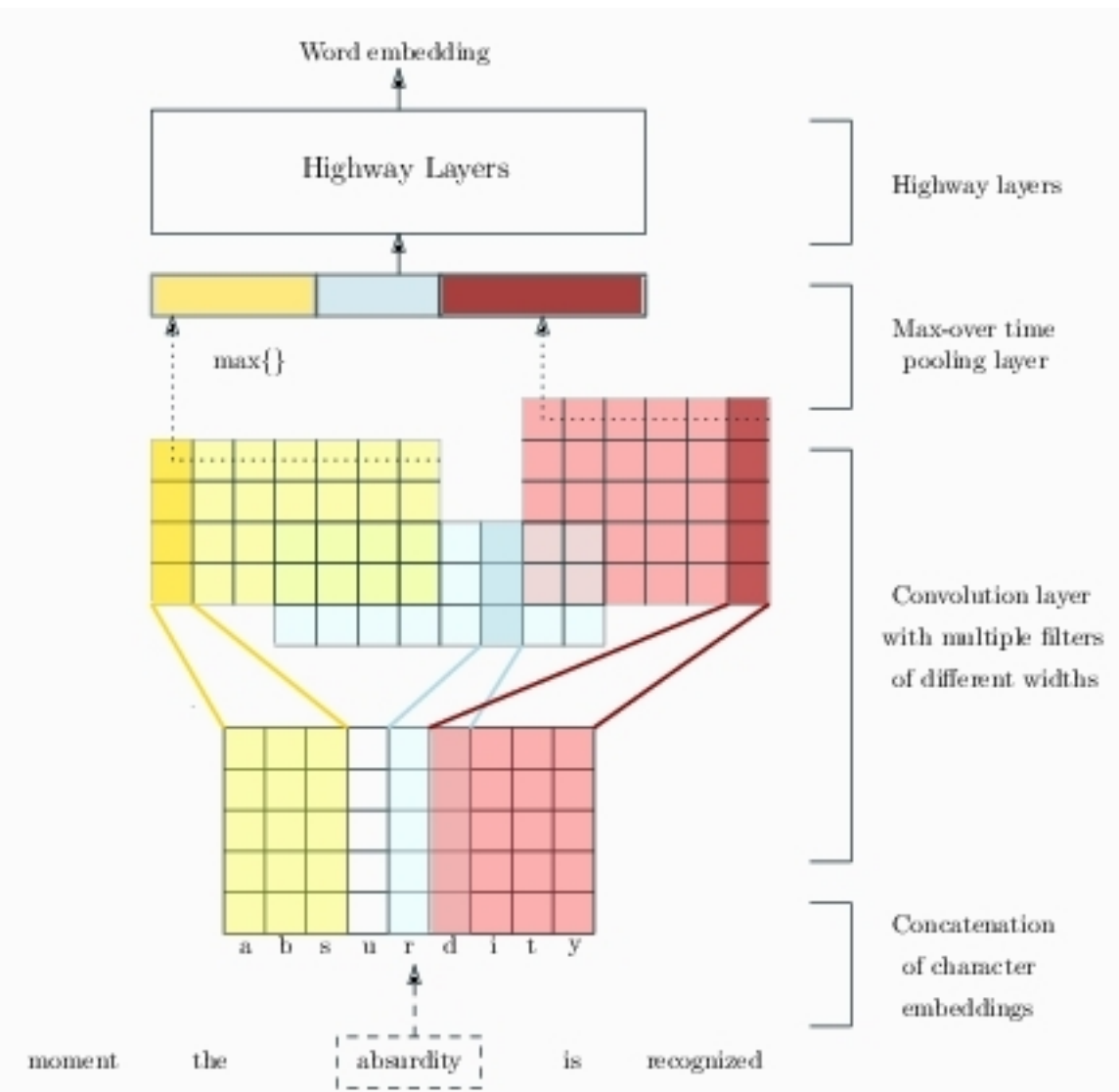
Convolutional character NMT (Lee 2017)



WMT'15 (Ge-En)

System	BLEU
BPE(s)+BPE(t) NN	24.0
BPE(s)+char(t) NN	25.3
char(s)+char(t) NN	25.8

Convolutional character encoder (Costa-Jussà & Fenollosa 2016)



System	BLEU	
	Ge-En	En-Ge
PBM	21.0	17.0
Word-based NN	20.6	17.2
Character-based NN	22.1	20.2

Refinements (Koehn, 2017)

- Ensemble decoding.
- Reranking n -best output list with a target language decoding.
- Large vocabularies and out-of-vocabulary words: BPE, categories, character-based translation, ...
- Adding monolingual data:
 - Back translation.
 - Adding a language model.
- Combining statistical alignment models and attention models.
- Adaptation.
- Adding linguistic annotation in the target.

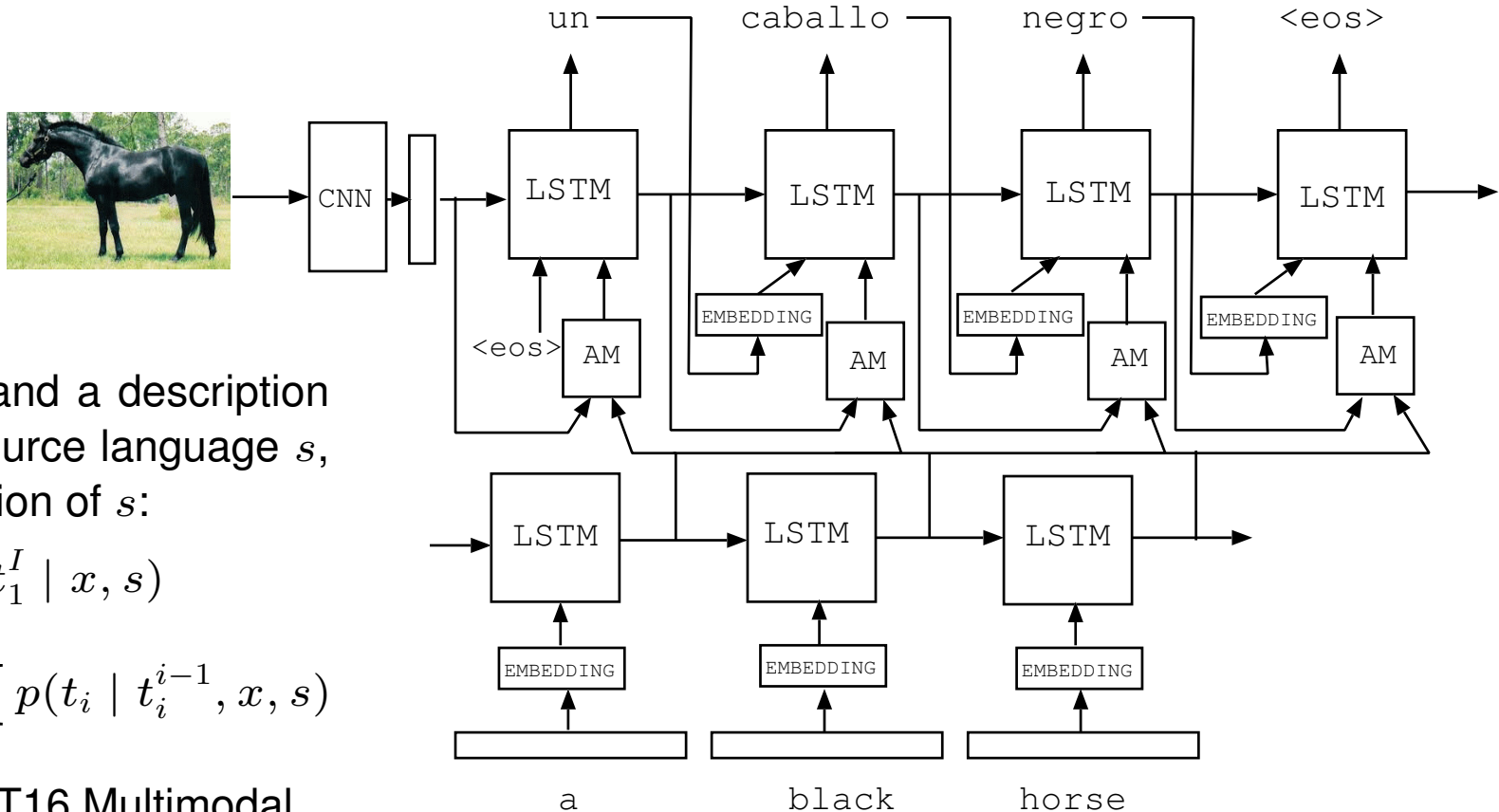
Other topics in NMT

- Multimodal machine translation (in this section)
- Speech machine translation (in this section)
- Computer-assisted translation (next topic)
- Context-aware machine translation
 - Cache-based
 - Additional encoders
- LegoNN: building encoder-decoder architectures with decoder modules that can be reused across various MT tasks [Damnia arXiv 2022]

Other topics in NMT

- Multilingual (source and/or target) machine translation.
- Translation with low-resource languages.
- Training NMT systems from monolingual corpora.
 - Back-translation
- Data selection (this is a general topic)
- The use of specific glossaries (place holders)
- Computational issues: computational graphs, parallelism, the use of GPUs, ...

Other topics in NMT: Multimodal machine translation (Huang 2016)



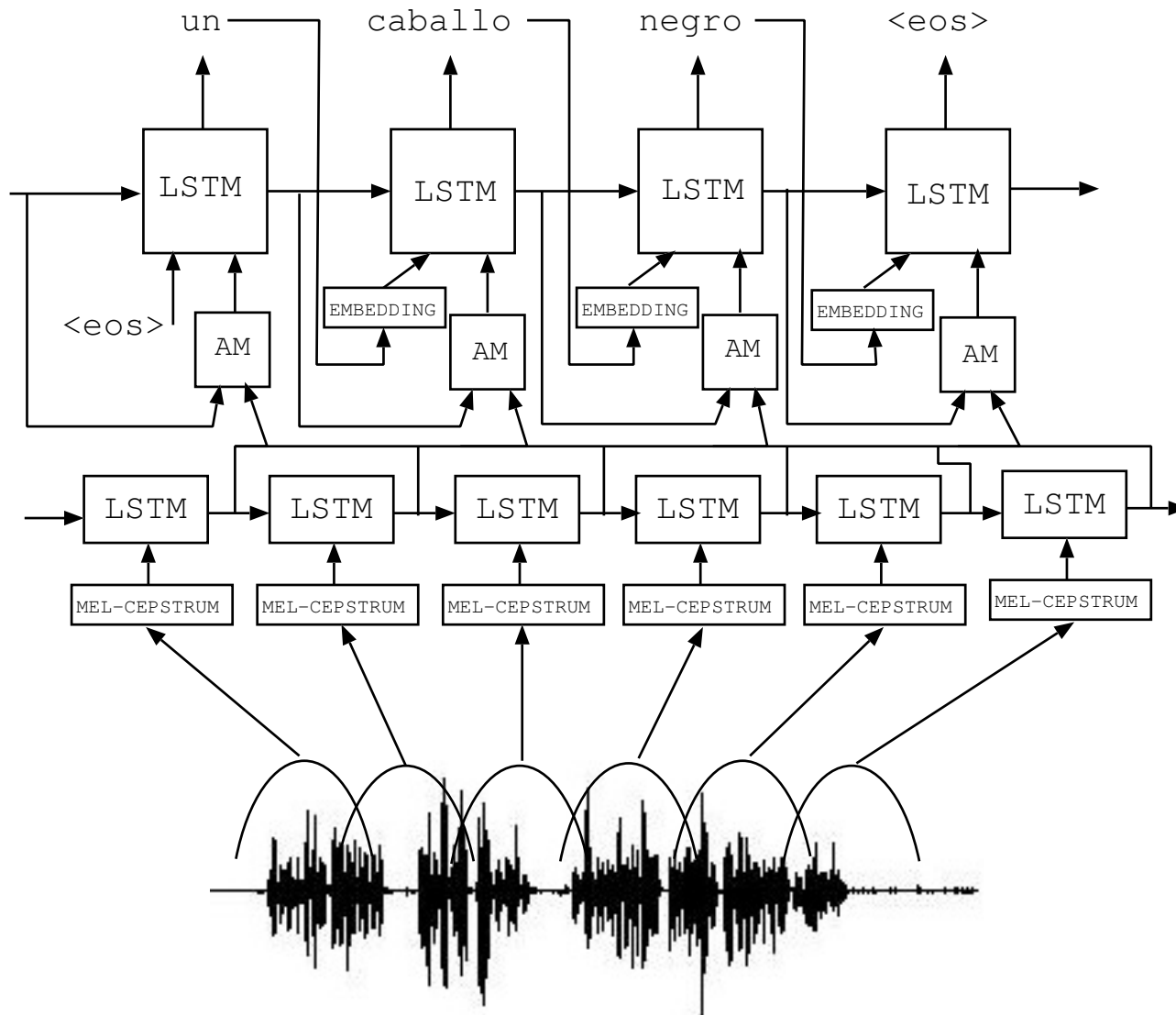
Given an image x and a description of the image in a source language s , search for a translation of s :

$$\begin{aligned}\hat{t}_1^I &= \operatorname{argmax}_{I,t} p(t_1^I | x, s) \\ &= \operatorname{argmax}_{I,t} \prod_{i=1}^I p(t_i | t_i^{i-1}, x, s)\end{aligned}$$

Experiments: WMT16 Multimodal machine translation
English-to-German

Systems	METEOR
Our model (without optimization)	41.1
Best system WMT16 (without Moses)	51.5
Best system WMT16 (using Moses)	53.2

Other topics in NMT: Speech machine translation (Weiss et al. 2017)



Given an utterance x , search for a translation of s :

$$\begin{aligned}\hat{t}_1^I &= \operatorname{argmax}_{I,t} p(t_1^I | x) \\ &= \operatorname{argmax}_{I,t} \prod_{i=1}^I p(t_i | t_i^{i-1}, x)\end{aligned}$$

Spanish-English Fisher task

Systems	BLEU
End-to-end	47.3
Supervised source	48.7
Cascade ASR+NMT	45.5

Toolkits for neural machine translation

- **Fairseq** <https://github.com/pytorch/fairseq>
Facebook, Google Brain (Ott+ arXiv 2019)
- **MarianNMT** <https://github.com/arian-nmt/arian/>
Microsoft, Adam Mickiewicz U., U. of Edinburgh, (Junczys-Dowmunt+ ACL 2018)
- **modernNMT** <https://github.com/modernmt/modernmt>
Facebook, FBK. Translated ()
- **Nematus** <https://github.com/EdinburghNLP/nematus>
U. of Edinburgh, Middle East U., New York U., Heidelberg U., U. of Zurich (Sennrich+ arXiv 2017)
- **NMT-Keras** <https://github.com/lvapeab/nmt-keras>
UPV (Peris+ PBML 2018)
- **OpenNMT** <https://github.com/opennmt>
SYSTRAN, Harvard U., Ubiquis (Klein+ arXiv 2018)
- **RETURNN** <https://github.com/rwth-i6/returnn>
RWTH Aachen, AppTek, NNAISENSE (Zeyer+ arXiv 2018)
- **Sockeye** <https://github.com/aws-labs/sockeye>
Amazon (Hieber+ arXiv 2017)
- **T2T** <https://github.com/tensorflow/tensor2tensor>
Google, DeepMind (Wasvani+ arXiv 2018)
- **XNMT** <https://github.com/neulab/xnmt>
CMU, KIT, Limsi, NIST, Inria, Pennsylvania U., Illinois U. (Neubig+ arXiv 2018)

Neural machine translation systems

- DeepL <https://www.deepl.com/en/translator>
- Google translate <https://translate.google.com/>
- Pure Neural Machine Translation <https://translate.systran.net/translationTools/text>
- UPV Neural Machine Translation <http://casmacat.prhlt.upv.es/inmt/>

Index

- 1 Introduction ▷ 2
- 2 Synchronized feedforward neural networks ▷ 11
- 3 Synchronized recurrent neural networks ▷ 14
- 4 Neural machine translation ▷ 23
- 5 Attention models and recurrent neural networks ▷ 31
- 6 Attention models for all ▷ 40
- 7 Training and Decoding with NMT ▷ 59
- 8 Translation results with neural machine translation ▷ 67
- 9 Other issues of NMT ▷ 73
- 10 *Bibliography* ▷ 84

Bibliography (1)

- Bahdanau et al. Neural machine translation by jointly learning to align and translate. arXiv:1409.0473. 2015
- Bengio et al. A Neural Probabilistic Language Model, JMLR 2003.
- Castaño & Casacuberta. Preliminary Experimentations for ASR Understanding Through Simple RN. Eurospeech 1995
- Castaño & Casacuberta. A Connectionist Approach to Machine translation. Eurospeech 1997.
- Castaño & Casacuberta. Text-to-text machine translation using the RECONTRA connectionist model. IWANN 1999.
- Chollampatt, Hendy Susanto, Tan, Szymanska. Can Automatic Post-Editing Improve NMT?. EMNLP 2020.
- Chung, Gulcehre, Cho, Bengio. Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling. arXiv:1412.3555. 2014
- Costa-Jusa & Fenollosa. Character-based neural machine translation. ACL. 2016.
- Elliot, Frank & Hasler. Multilingual Image Description with Neural Sequence Models. ICLR. 2016.
- Hochreiter & Schmidhuber. Long short-term memory. Neural Computation. 1997.
- Koehn. Neural Machine Translation. arXiv:1709.07809v1. 2017.
- Koehn. Neural Machine Translation. Cambridge Univ. 2020.

Bibliography (2)

- Kratzer, Toussaint, Mainprice. Prediction of Human Full-Body Movements with Motion Optimization and Recurrent Neural Networks. IEEE ICRA. 2020.
- Lakew et al. A Comparison of Transformer and Recurrent Neural Networks on Multilingual Neural Machine Translation. COLING 2018.
- Larriba. Character-based Neural Machine Translation. Seminar PRHLT. 2018.
- Lee et al. Fully Character-Level Neural Machine Translation without Explicit Segmentation. TACL. 2017.
- Luong et al. Effective Approaches to Attention-based Neural Machine Translation. arXiv:1508.04025. 2015.
- Mikolov et al. Distributed Representations of Words and Phrases and their Compositionality. arXiv:1310.4546. 2013
- Mikolov et al. Distributed Representations of Words and Phrases and their Compositionality. NIPS 2013.
- Ney. Speech Recognition and Machine Translation: From Statistical Decision Theory to Machine Learning and Deep Neural Networks. 2nd International Summer School on Deep Learning. 2018
- Gers & Schmidhuber. Recurrent nets that time and count. Proceedings of the IEEE-INNS-ENNS International Joint Conference on Neural Networks.2000.

Bibliography (3)

- Peris. Interactivity, adaptation and multimodality in neural sequence-to-sequence learning. Ph.D. Thesis. 2019.
- Peris et al. Video Description Using Bidirectional Recurrent Neural Networks. ICANN. 2016
- Peris & Casacuberta. Interactive neural machine translation. CSL. 2017.
- Ruder. An overview of gradient descent optimization algorithms. arXiv, 2016
- Schuster & Paliwal. Bidirectional Recurrent Neural Networks. IEEE Transactions on Signal Processing. 1997.
- Sennrich et al. Neural Machine Translation of Rare Words with Subword Units. arXiv:1508.07909. 2015.
- Sennrich et al. Advances in Neural Machine Translation. AMTA. 2016.
- Sundermeyer. Improvements in Language and Translation Modeling, PhD. 2015
- Sutskever et al. Sequence to Sequence Learning with Neural Networks. arXiv 2014.
- Vinyals et al. Show and Tell: A Neural Image Caption Generator. CVPR. 2015
- Vaswani et al. Attention Is All You Need. arXiv 2017.
- Weiss et al. Sequence-to-Sequence Models Can Directly Translate Foreign Speech. arXiv 2017.
- Yao et al. Describing Videos by Exploiting Temporal Structure. arXiv:1502.08029.2015.